

Final Semester Exam for Database Systems

Girls' Boarding House Information System



Lecturer by:

Asmunin, S.Kom., M.Kom.

Written by :

Aisyah Currents

25091397108

International Class 2025

INFORMATIC MANAGEMENT MAJOR

FACULTY OF VOCATION

UNIVERSITAS NEGERI SURABAYA

YEAR 2026



TABLE OF CONTENTS

TABLE OF CONTENTS	2
TABLE OF FIGURES	3
A. BUSINESS PROCESS AND MODULES.....	5
B. THE ENTITIES	6
C. ENTITY RELATIONSHIP DIAGRAM	7
D. THE CARDINALITY	8
E. DATABASE NORMALIZATION PERFORM	8
F. THE NORMALIZED DATABASE DESIGN IN MYSQL	9
G. SQL COMMANDS.....	14
H. CRUD OPERATIONS ON 2 SELECTED MASTER TABLES	20
I. GITHUB LINK.....	45

TABLE OF FIGURES

Figure 1. Business Process Diagram design using draw.io	5
Figure 2. ED design using ERDPlus.....	7
Figure 3. Create Database	10
Figure 4. Showing All Databases and Succeed	10
Figure 5. Create table Master Data tenant.....	11
Figure 6. Create table Master Data room	11
Figure 7. Create table Master Data facility	12
Figure 8. Create table Transaction Data payment.....	12
Figure 9. Create table Report Data history	13
Figure 10. Show the Database Diagram.....	13
Figure 11. Query Insert Data table tenant	14
Figure 12. Result Insert Data table tenant	14
Figure 13. Query Insert Data table room.....	15
Figure 14. Result Insert Data table room	15
Figure 15. Query Insert Data table facility	16
Figure 16. Result Insert Data table facility.....	16
Figure 17. Query Insert Data table payment.....	16
Figure 18. Result Insert Data table payment	17
Figure 19. Query Insert Data table history	17
Figure 20. Result Insert Data table history	17
Figure 21. Query Update Data table room	18
Figure 22. Result Update Data table room	18
Figure 23. Query Update Data table facility.....	19
Figure 24. Result Update Data table facility	19
Figure 25. Query Delete Data table history	19
Figure 26. Result Delete Data table history.....	19
Figure 27. Query Delete Data table facility.....	20
Figure 28. Result Delete Data table facility	20
Figure 29. Home page Python Application	21
Figure 30. Tenant page Python Application.....	22
Figure 31. Add tenant Python Application.....	23
Figure 32. Search tenant Python Application	24
Figure 33. Edit tenant Python Application.....	25
Figure 34. Delete tenant Python Application	26
Figure 35. Room page Python Application	27
Figure 36. Add room Python Application	28
Figure 37. Search room Python Application	29
Figure 38. Edit room Python Application	30
Figure 39. Delete room Python Application.....	31

Figure 40. Dashboard PHP Application.....	32
Figure 41. Tenant pages PHP Application	33
Figure 42. Add tenant PHP Application	34
Figure 43. Search tenant PHP Application.....	36
Figure 44. Edit tenant PHP Application.....	37
Figure 45. Delete tenant PHP Application	38
Figure 46. Room pages PHP Application	39
Figure 47. Add room PHP Application.....	40
Figure 48. Search room PHP Application	41
Figure 49. Edit room PHP Application.....	42
Figure 50. Delete room PHP Application	44

A. BUSINESS PROCESS AND MODULES

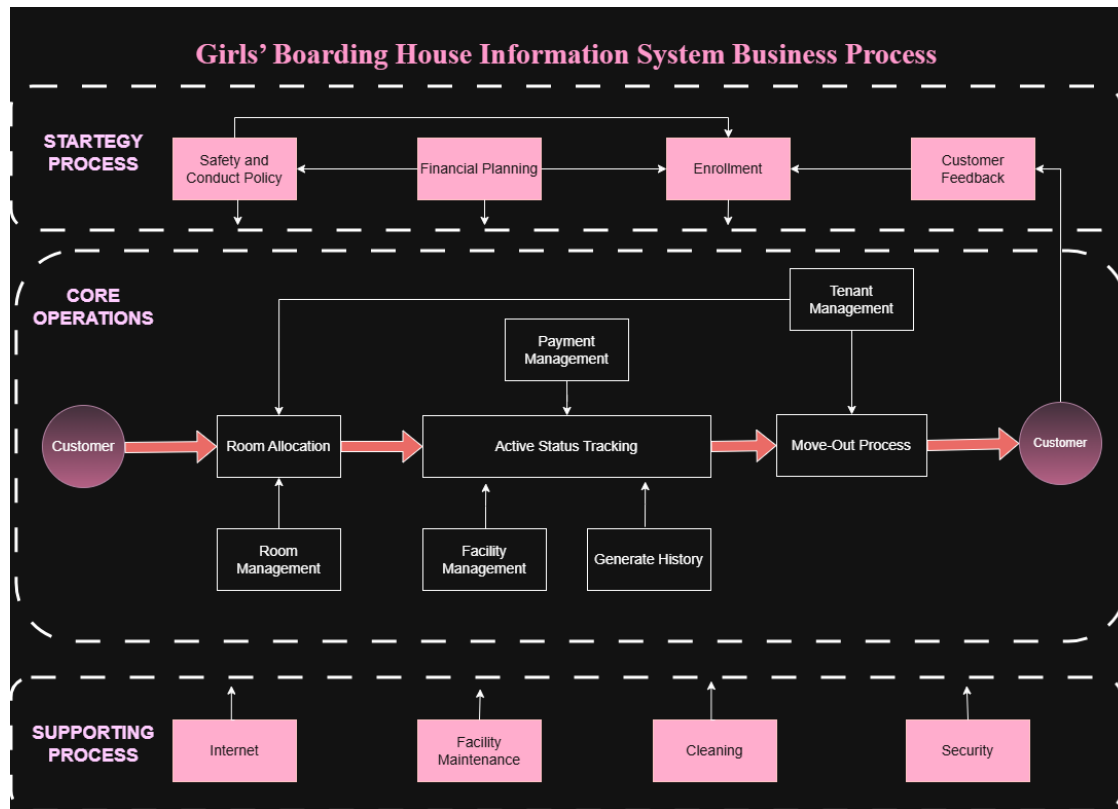


Figure 1. Business Process Diagram design using draw.io

The business process begins with strategic planning activities such as creating safety and conduct policies, financial planning, tenant enrollment, and collecting customer feedback. After registration, customers enter the core operational process where rooms are allocated based on availability. Room allocation is supported by room management activities. During the tenant's stay, the system monitors active status tracking, payment management, facility management, and generates tenant history records. When tenants finish their rental period, the move-out process is conducted through tenant management procedures. Supporting processes such as internet services, facility maintenance, cleaning services, and security ensure that boarding house operations run effectively for tenants. Modules included in the Business Process are :

1. Room Management

The Room Management module is used to manage room data, including room availability, room types, room prices, and room conditions. This module helps administrators monitor which rooms are available or occupied.

2. Tenant Management

The Tenant Management module manages tenant information such as personal data, contact information, room assignments, and tenant status during their stay in the boarding house.

3. Facility Management

The Facility Management module manages boarding house facilities such as WiFi, kitchen areas, laundry services, cleaning services, and security facilities to ensure tenant comfort and operational efficiency.

4. Payment Management

The Payment Management module is responsible for recording rental payments, monitoring payment status, and managing tenant transaction records for each payment period.

5. Report History

The Report History module generates reports related to tenant activities, payment history, room occupancy, and other operational data that help administrators monitor and evaluate boarding house operations.

B. THE ENTITIES

The Girls' Boarding House Information System consists of several important modules that support boarding house operations and management activities. Each module contains entities that store and manage specific data required by the system. These entities are interconnected to help administrators organize tenant information, room availability, payment transactions, facility data, and operational reports efficiently. The following are the entities involved in each module of the system.

1. Tenant Management

m_tenant (tenant_id, name, phone, address, emergency_contact), The Tenant entity is used to store tenant personal information such as tenant name, phone number, address, and emergency contact. This entity helps administrators manage and monitor all tenant data in the boarding house system.

2. Room Management

m_room (room_id, room_number, type, price, status), The Room entity stores information related to boarding house rooms, including room number, room type, room price, and room availability status. This entity helps administrators manage room occupancy and room conditions.

3. Facility Management

m_facility (facility_id, name, condition, description), The Facility entity stores information about facilities provided by the boarding house, such as WiFi, kitchen, laundry room, and parking area. It also records facility conditions and descriptions for maintenance purposes.

4. Payment Management

t_payment (payment_id, tenant_id, room_id, amount, payment_date, status), The Payment entity records all payment transactions made by tenants. It stores information such as payment amount, payment date, room rented, and payment status to help manage financial records and monitor monthly rent payments.

5. Report History

r_history (report_id, tenant_id, room_id, payment_id, report_date, description), The ReportHistory entity is used to generate and store reports related to tenant activities, payment history, and room occupancy. This entity helps administrators monitor boarding house operations and maintain historical records.

C. ENTITY RELATIONSHIP DIAGRAM

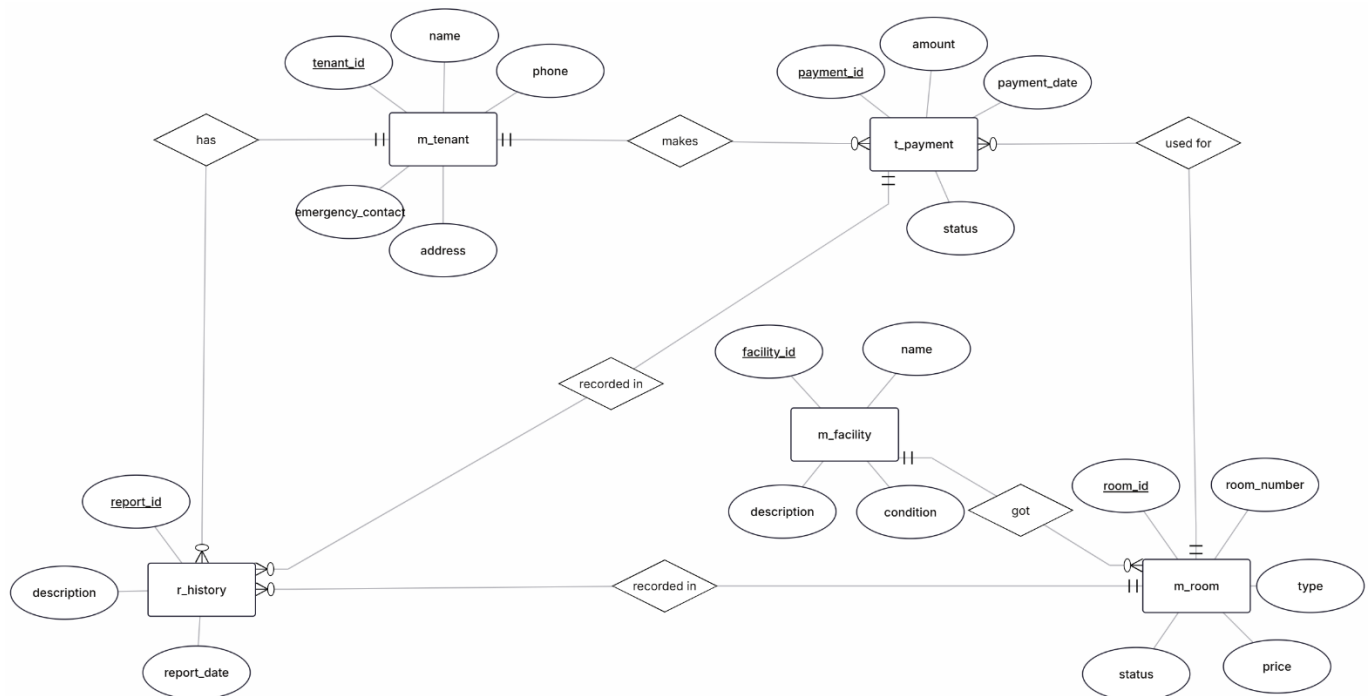


Figure 2. ERD design using ERDPlus

The Entity Relationship Diagram (ERD) of the Girls' Boarding House Information System is designed to represent the relationships between entities involved in boarding house management activities. The system consists of five main entities: **m_tenant**, **m_room**, **m_facility**, **t_payment**, and **r_history**. The **m_tenant** entity stores tenant information such as tenant ID, name, phone number, address, and emergency contact. The **m_room** entity stores room information including room number, room type, room status, and room price. The **m_facility** entity stores facility data such as facility name, condition, and description. Meanwhile, the **t_payment** entity records payment transactions including payment amount, payment date, and payment status, while the **r_history** entity stores operational history and report data related to boarding house activities.

The relationships between entities are designed to support the operational process of the boarding house system. The relationship between **m_tenant** and **t_payment** shows that one tenant can make multiple payments, while each payment belongs to one tenant. The relationship between **m_room** and **t_payment** indicates that one room can be associated with many payment transactions over time. The **r_history** entity is connected to **m_tenant**, **m_room**, and **t_payment** to store historical records related to tenant activities, room

occupancy, and payment transactions. In addition, the relationship between m_room and m_facility shows that one room can have several facilities. Overall, the ERD illustrates how all entities are interconnected to support efficient and organized boarding house management.

D. THE CARDINALITY

Cardinality is used to describe the number of relationships between one entity and another entity in the database system. The cardinality of each relationship in the Girls' Boarding House Information System is as follows:

1. Makes (m_tenant - t_payment)
One tenant can make many payment transactions, while each payment transaction belongs to only one tenant so it is One-to-Many (1 : N).
2. Used for (m_room - t_payment)
One room can be associated with many payment transactions over time, while each payment transaction is related to only one room so it is One-to-Many (1 : N).
3. Has (m_tenant - r_history)
One tenant can have many history records, while each history record belongs to only one tenant so it is One-to-Many (1 : N).
4. Recorded in (m_room - r_history)
One room can appear in many history records, while each history record refers to only one room so it is One-to-Many (1 : N).
5. Recorded in (t_payment - r_history)
One payment transaction can be recorded in many history reports, while each history report references only one payment transaction so it is One-to-Many (1 : N).
6. Got (m_room - m_facility)
One room can have several facilities, while each facility belongs to only one room so it is One-to-Many (1 : N).

E. DATABASE NORMALIZATION PERFORM

Database normalization is performed to organize the database structure efficiently and reduce data redundancy. The normalization process in the Girls' Boarding House Information System is performed up to the Third Normal Form (3NF).

1. First Normal Form (1NF)

The database satisfies the First Normal Form because each table contains atomic values and there are no repeating groups or duplicate attributes. Each entity has a primary key that uniquely identifies every record.

Table	Primary Key	Atomic Attributes
m_tenant	tenant_id	name, phone, emergency_contact, address
t_payment	payment_id	amount, payment_date, status
m_room	room_id	room_number, type, price, status
m_facility	facility_id	name, description, condition
r_history	report_id	description, report_date

2. Second Normal Form (2NF)

The database satisfies the Second Normal Form because all non-key attributes are fully dependent on the primary key. There are no partial dependencies in the tables.

Table	Primary Key	Non-Key Attributes	Dependency
m_tenant	tenant_id	name, phone, emergency_contact, address	Fully dependent on tenant_id
t_payment	payment_id	amount, payment_date, status	Fully dependent on payment_id
m_room	room_id	room_number, type, price, status	Fully dependent on room_id
m_facility	facility_id	name, description, condition	Fully dependent on facility_id
r_history	report_id	description, report_date	Fully dependent on report_id

3. Third Normal Form (3NF)

The database satisfies the Third Normal Form because there are no transitive dependencies between non-key attributes. Every non-key attribute depends only on the primary key of its table. This normalization process helps improve data consistency, reduce redundancy, and maintain database integrity.

Table	Primary Key	Non-Key Attributes	Transitive Dependency
m_tenant	tenant_id	name, phone, emergency_contact, address	None
t_payment	payment_id	amount, payment_date, status	None
m_room	room_id	room_number, type, price, status	None
m_facility	facility_id	name, description, condition	None
r_history	report_id	description, report_date	None

F. THE NORMALIZED DATABASE DESIGN IN MYSQL

The normalized database design of the Girls' Boarding House Information System is implemented using MySQL DBMS. The implementation includes the creation of tables, primary keys, foreign keys, and relationships between entities based on the normalized ERD design.

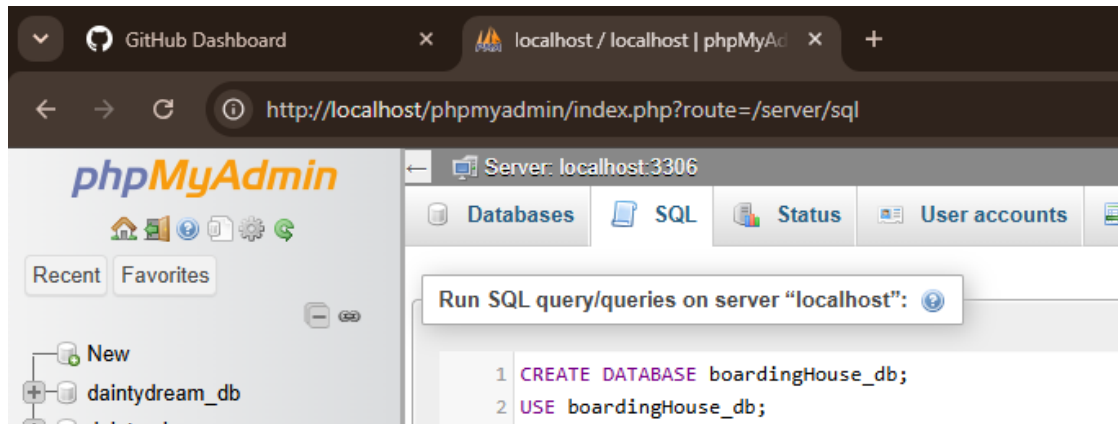


Figure 3. Create Database

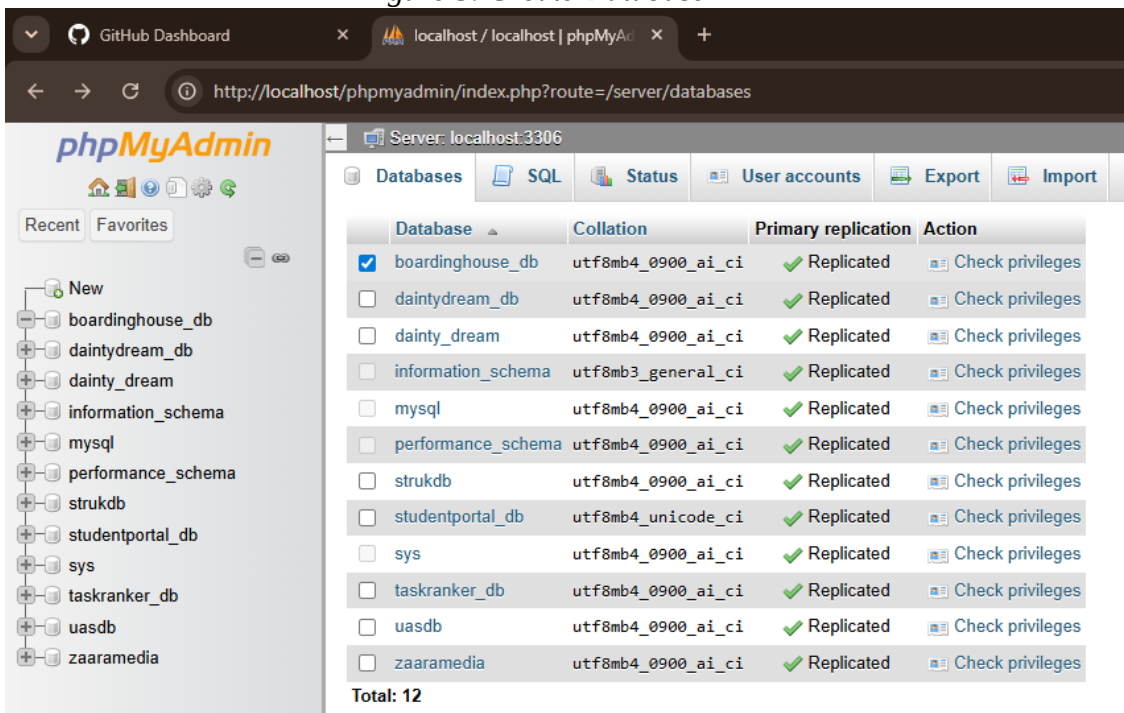


Figure 4. Showing All Databases and Succeed

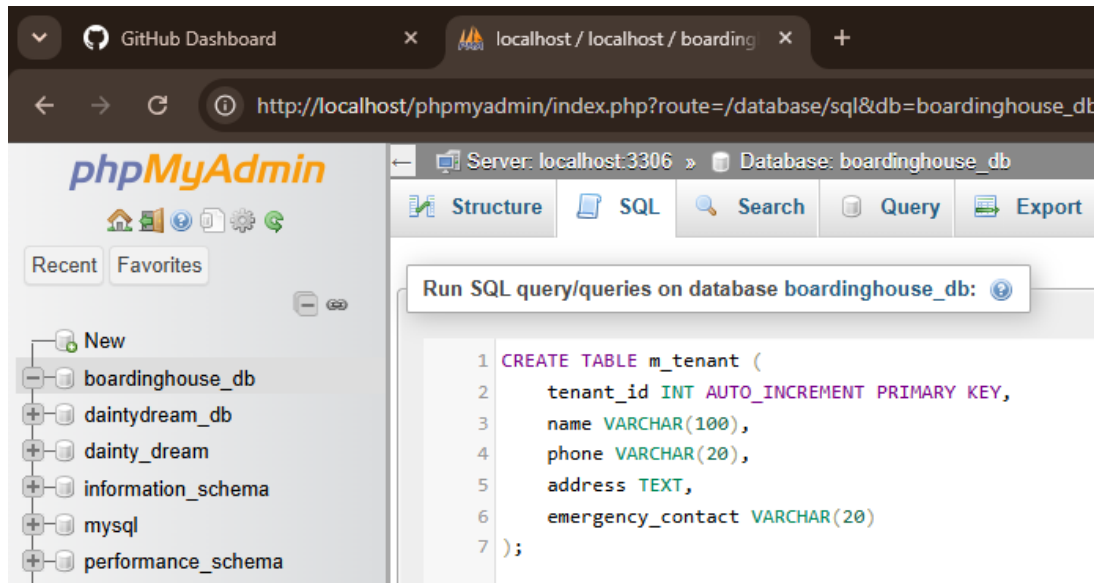


Figure 5. Create table Master Data tenant

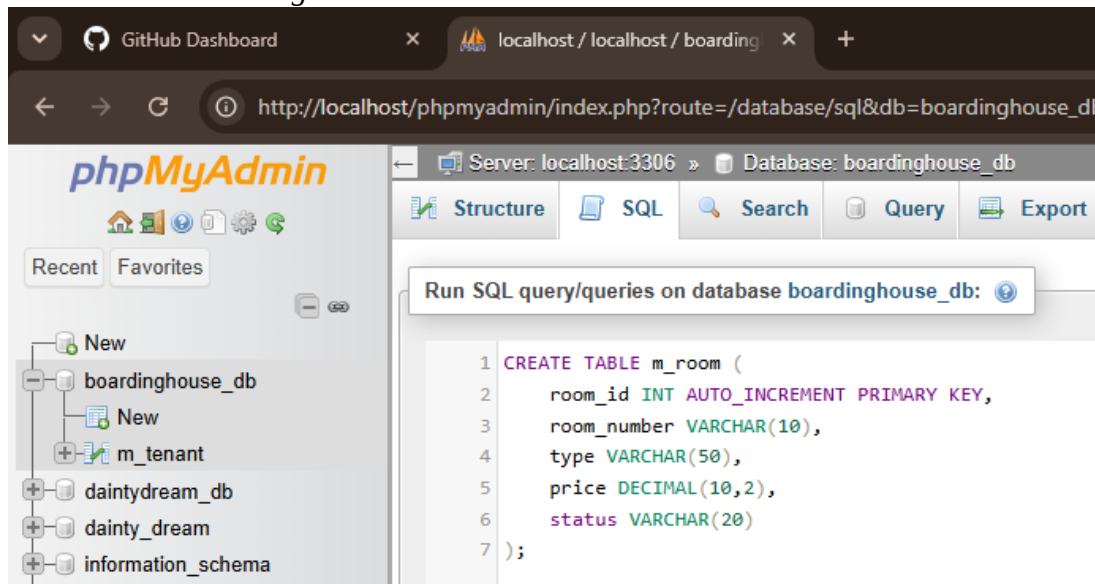


Figure 6. Create table Master Data room

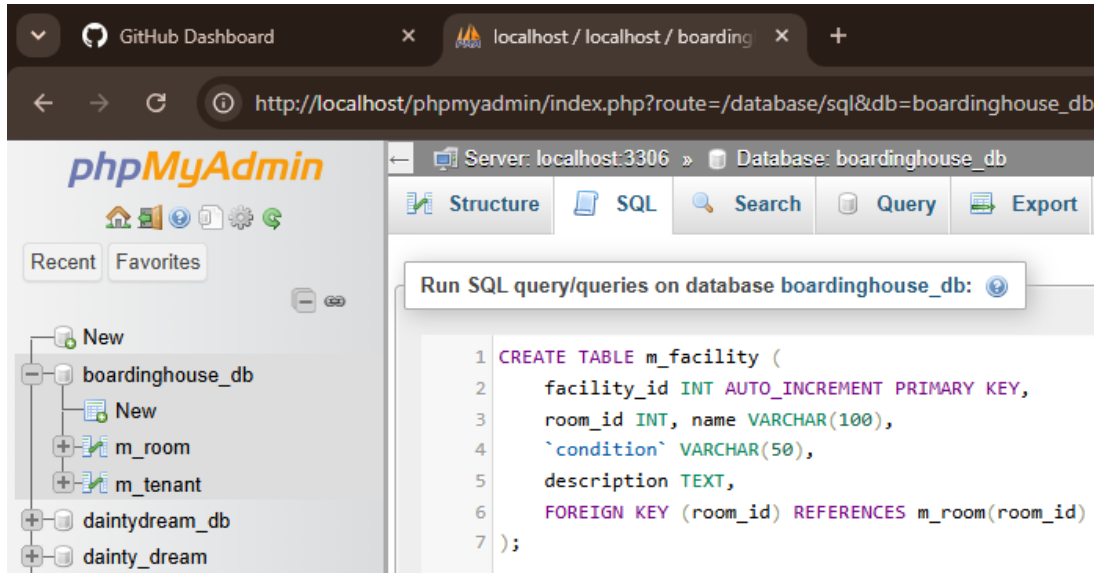


Figure 7. Create table Master Data facility

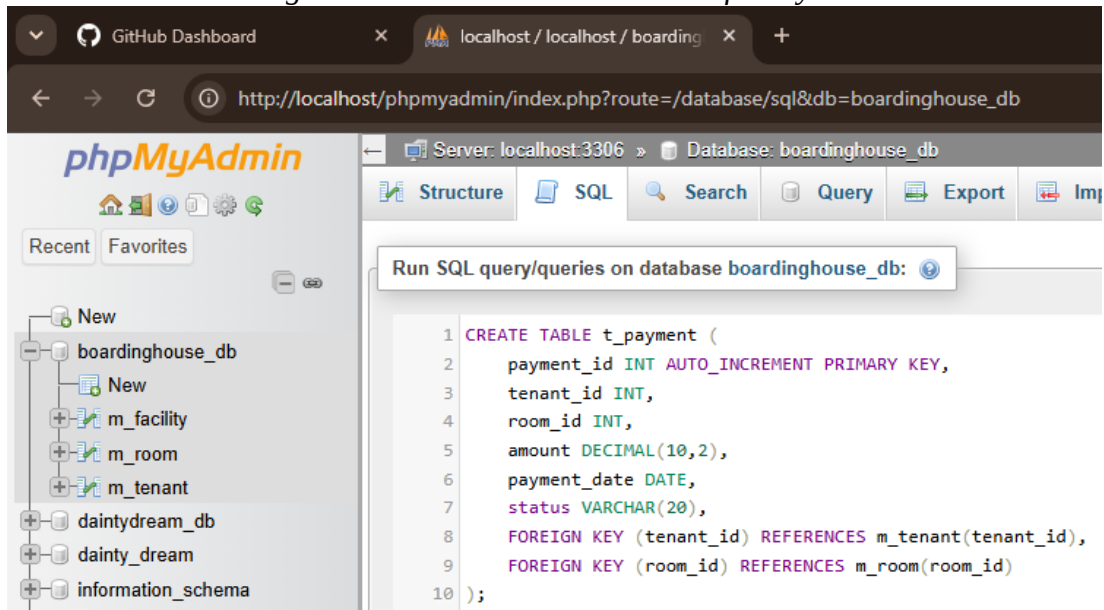


Figure 8. Create table Transaction Data payment

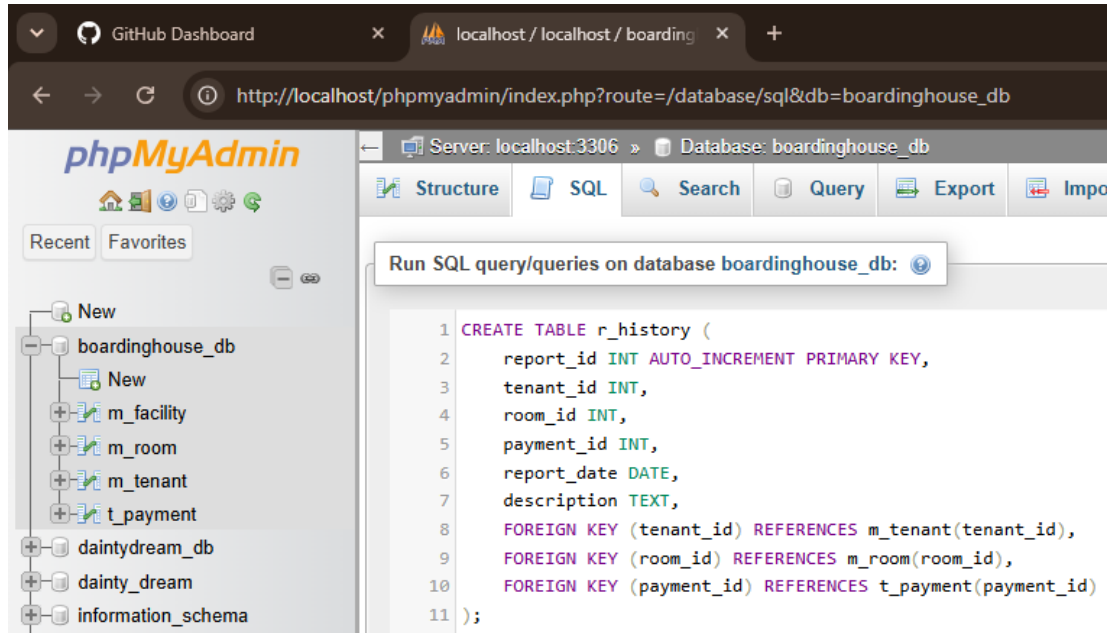


Figure 9. Create table Report Data history

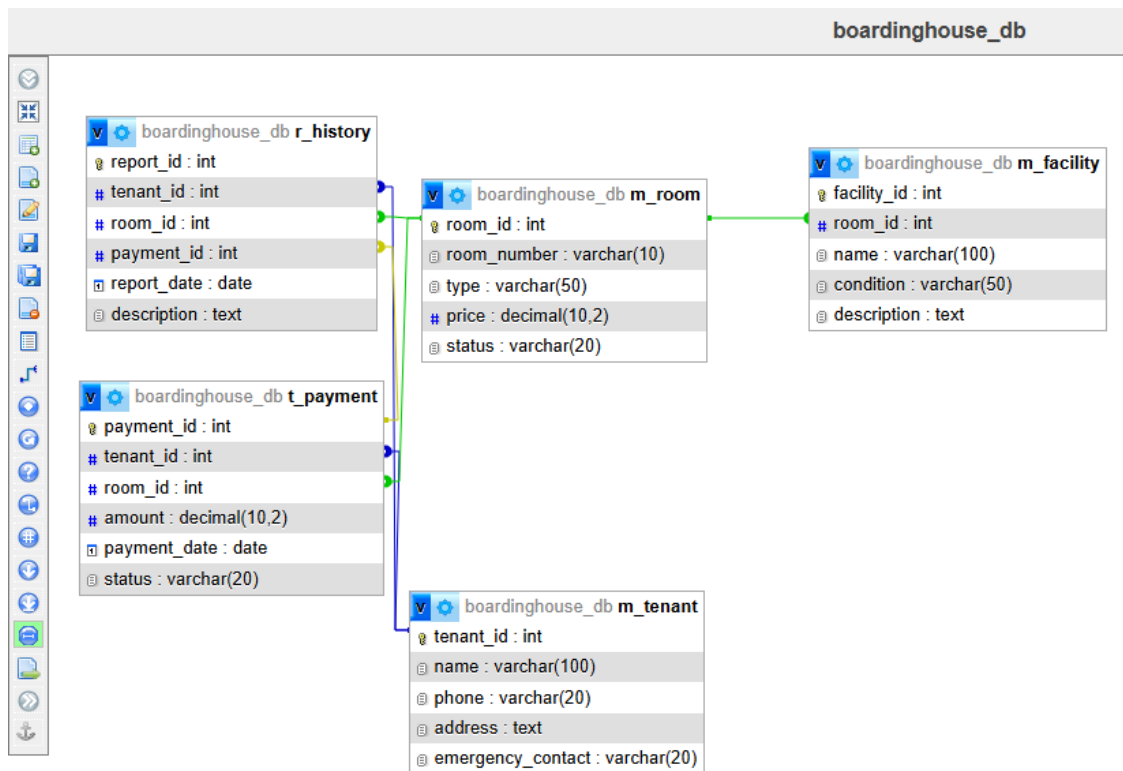


Figure 10. Show the Database Diagram

G. SQL COMMANDS

The following SQL commands are used to insert, update, and delete data from the database tables.

1. Insert Data

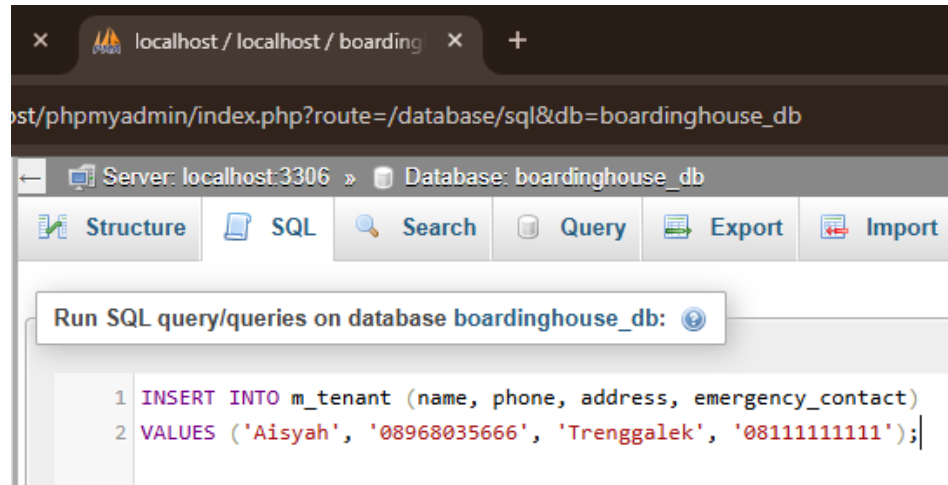


Figure 11. Query Insert Data table tenant

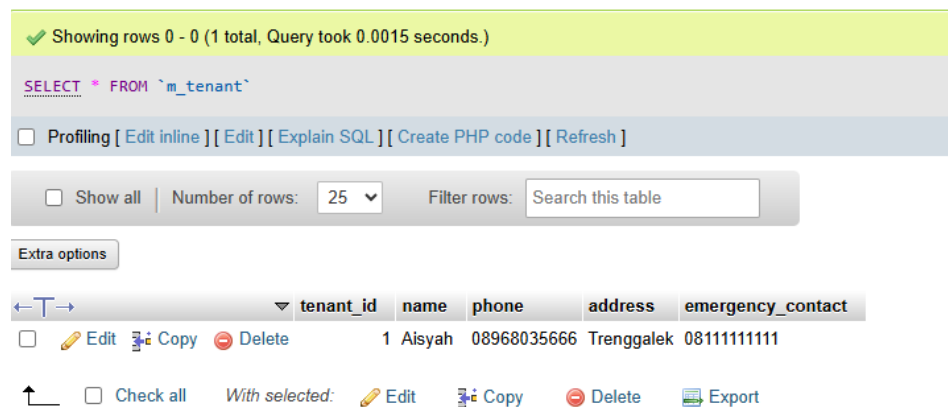


Figure 12. Result Insert Data table tenant

The following query is used to insert tenant data into the m_tenant table. The inserted data includes tenant name, phone number, address, and emergency contact information.

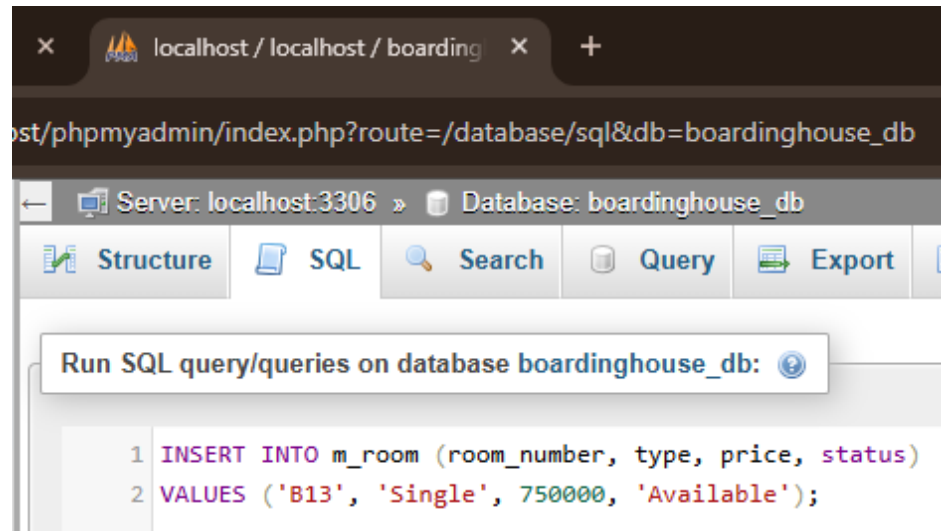


Figure 13. Query Insert Data table room

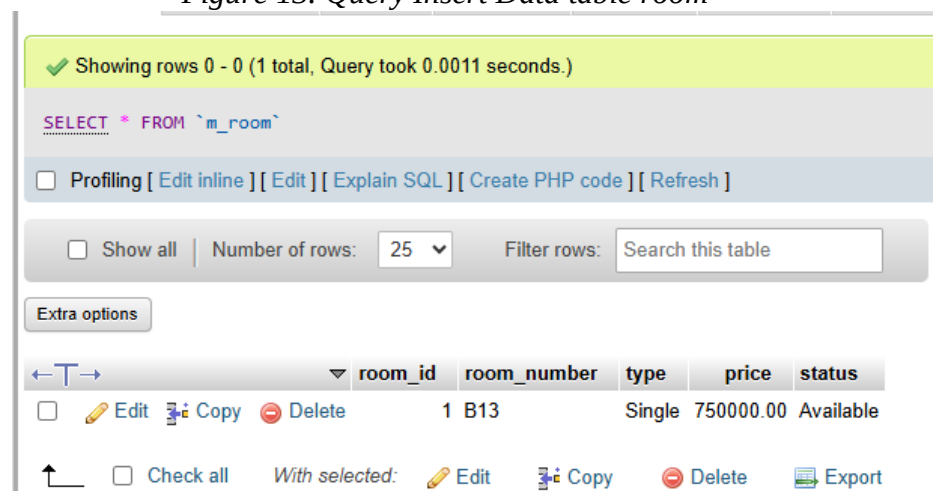


Figure 14. Result Insert Data table room

The following query is used to insert room data into the m_room table. The data includes room number, room type, room price, and room availability status.

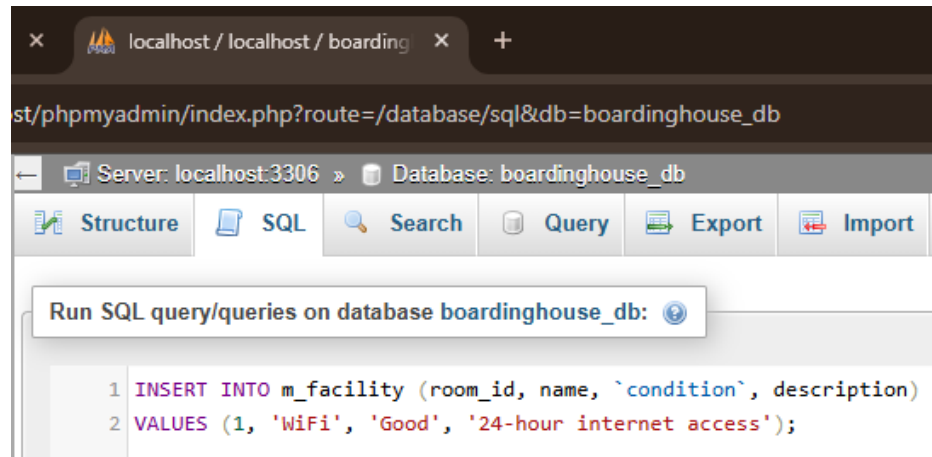


Figure 15. Query Insert Data table facility

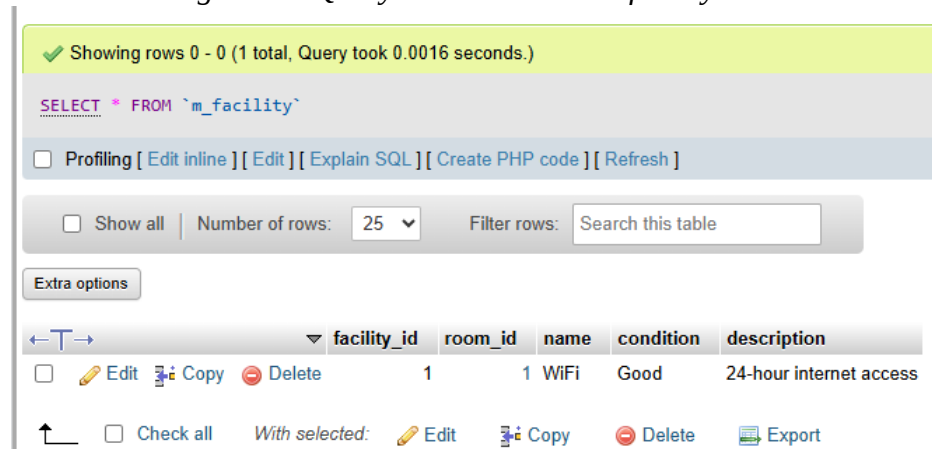


Figure 16. Result Insert Data table facility

The following query is used to insert facility data into the m_facility table. The inserted data includes the room ID, facility name, facility condition, and facility description.

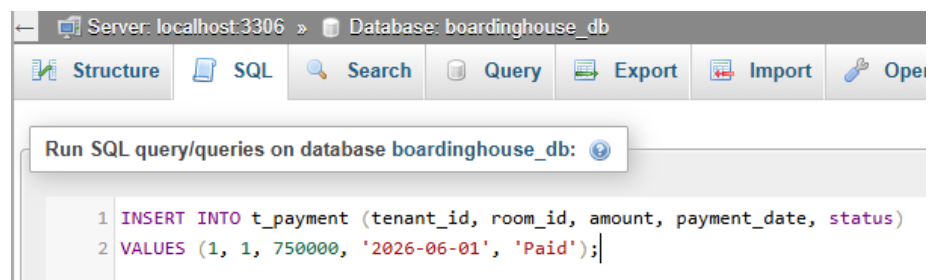


Figure 17. Query Insert Data table payment

Showing rows 0 - 0 (1 total, Query took 0.0016 seconds.)

`SELECT * FROM `t_payment``

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

	payment_id	tenant_id	room_id	amount	payment_date	status
☐ Edit Copy Delete	1	1	1	750000.00	2026-06-01	Paid

Check all | With selected: Edit Copy Delete Export

Figure 18. Result Insert Data table payment

The following query is used to insert payment transaction data into the t_payment table. The data includes tenant ID, room ID, payment amount, payment date, and payment status.

localhost / localhost / boarding

st/phpmyadmin/index.php?route=/database/sql&db=boardinghouse_db

Server: localhost:3306 Database: boardinghouse_db

Structure SQL Search Query Export Import Operations

Run SQL query/queries on database boardinghouse_db:

```

1 INSERT INTO r_history (tenant_id, room_id, payment_id, report_date, description)
2 VALUES (1, 1, 1, '2026-06-01', 'Monthly payment completed');

```

Figure 19. Query Insert Data table history

Showing rows 0 - 0 (1 total, Query took 0.0024 seconds.)

`SELECT * FROM `r_history``

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

	report_id	tenant_id	room_id	payment_id	report_date	description
☐ Edit Copy Delete	1	1	1	1	2026-06-01	Monthly payment completed

Check all | With selected: Edit Copy Delete Export

Figure 20. Result Insert Data table history

The following query is used to insert operational history data into the r_history table. The inserted data includes tenant ID, room ID, payment ID, report date, and report description.

2. Update Data

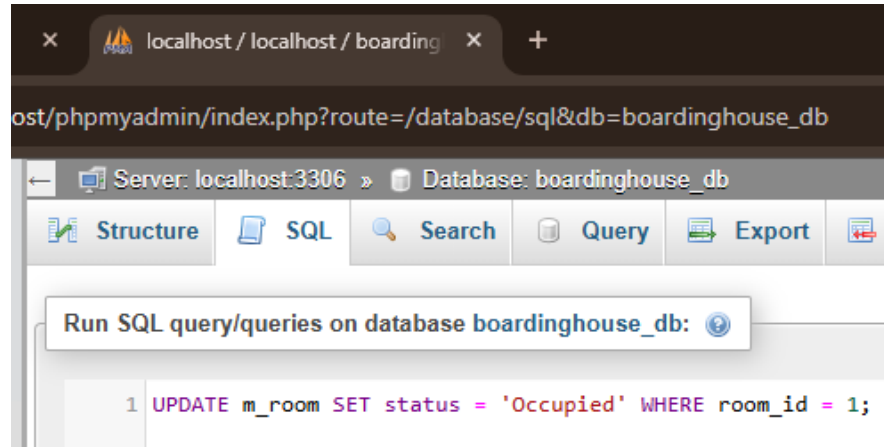


Figure 21. Query Update Data table room

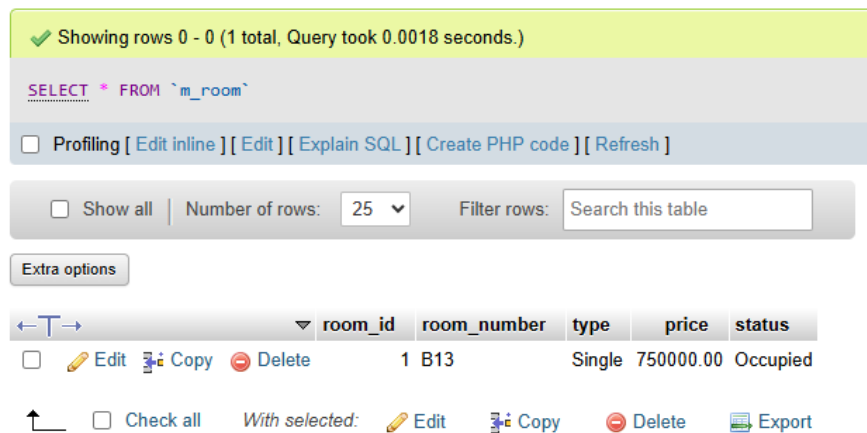


Figure 22. Result Update Data table room

The following query is used to update room status data in the m_room table. In this example, the room status is changed from “Available” to “Occupied”.

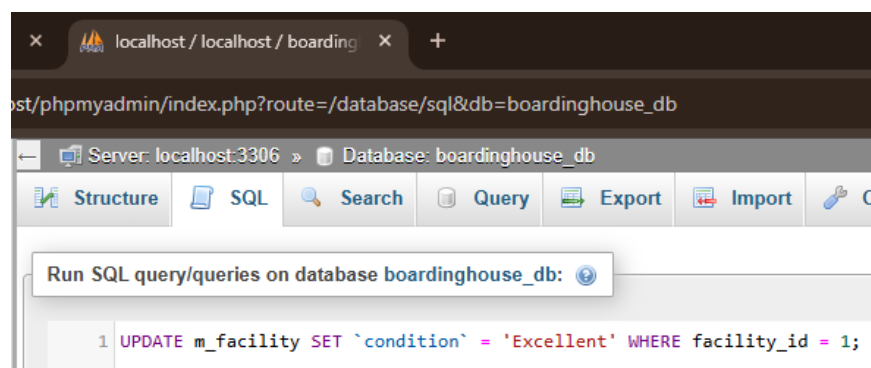


Figure 23. Query Update Data table facility

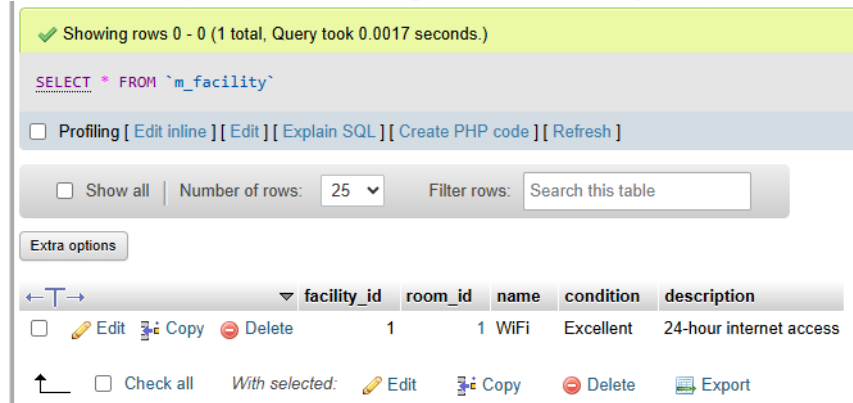


Figure 24. Result Update Data table facility

The following query is used to update facility condition data in the m_facility table.
The facility condition is updated to “Excellent”.

3. Delete Data

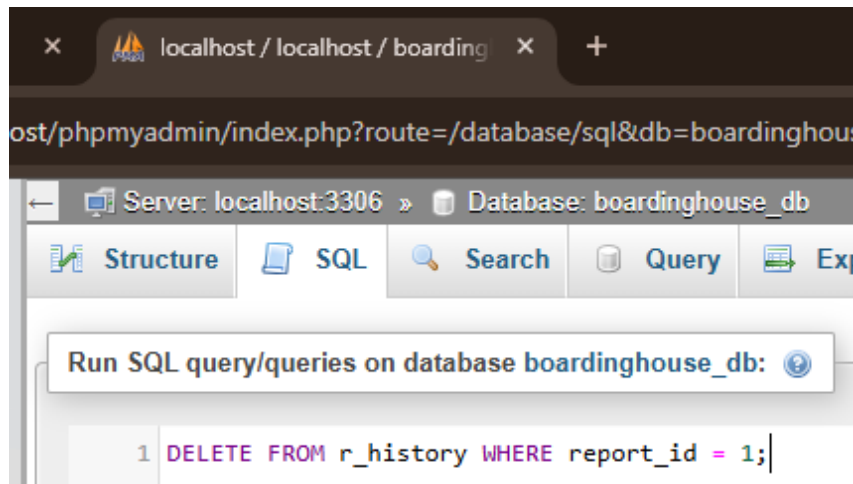


Figure 25. Query Delete Data table history

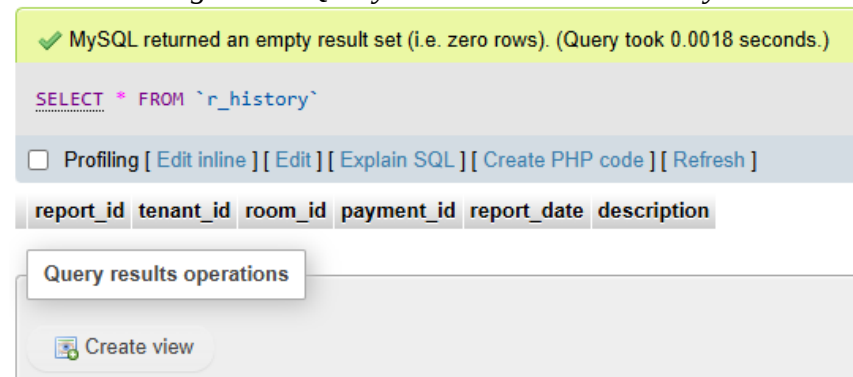


Figure 26. Result Delete Data table history

The following query is used to delete report history data from the `r_history` table based on the report ID.

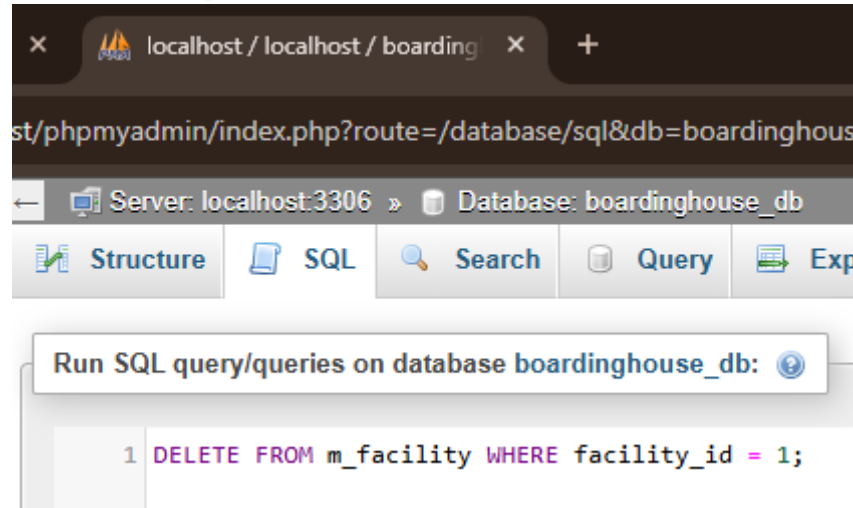


Figure 27. Query Delete Data table facility

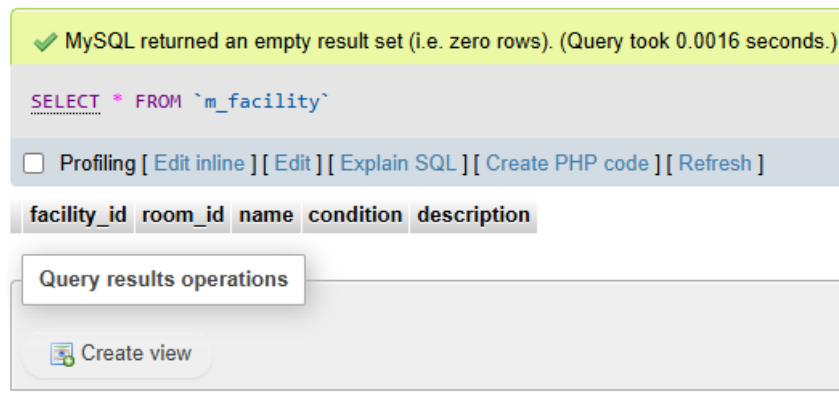


Figure 28. Result Delete Data table facility

The following query is used to delete facility data from the `m_facility` table based on the facility ID.

H. CRUD OPERATIONS ON 2 SELECTED MASTER TABLES

The CRUD operations in the Girls' Boarding House Information System are implemented on two selected master tables, namely `m_tenant` and `m_room`. CRUD stands for Create, Read, Update, and Delete, which are the four fundamental operations used in database management systems. These operations are essential for managing and maintaining data within the application effectively and efficiently. Through the implementation of CRUD operations, administrators are able to add new data, display stored information, modify existing records, and remove unnecessary data from the system. The application was developed using Python and PHP as the main programming languages,

while MySQL is used as the database management system to store all related information securely.

The CRUD operations implemented in this system greatly assist administrators in managing boarding house data in a more organized and systematic manner. In addition, these operations improve data accuracy, reduce manual recording errors, and increase the efficiency of daily administrative tasks. Therefore, the implementation of CRUD operations becomes one of the most important components in supporting the functionality and performance of the Girls' Boarding House Information System.

1. Python Application

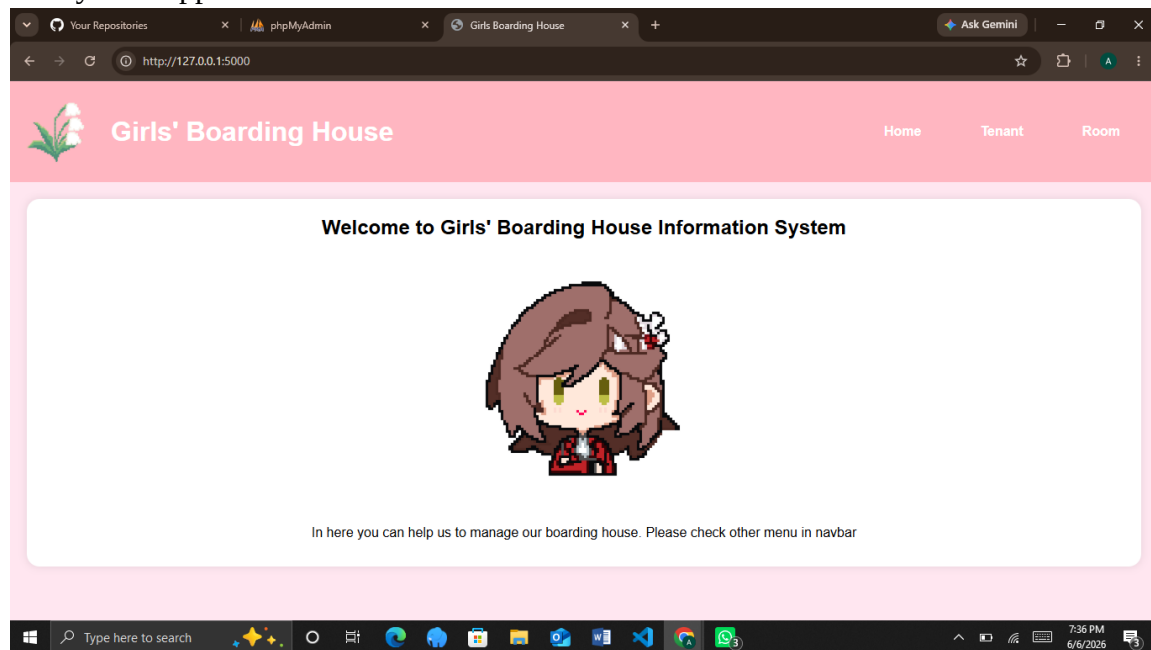
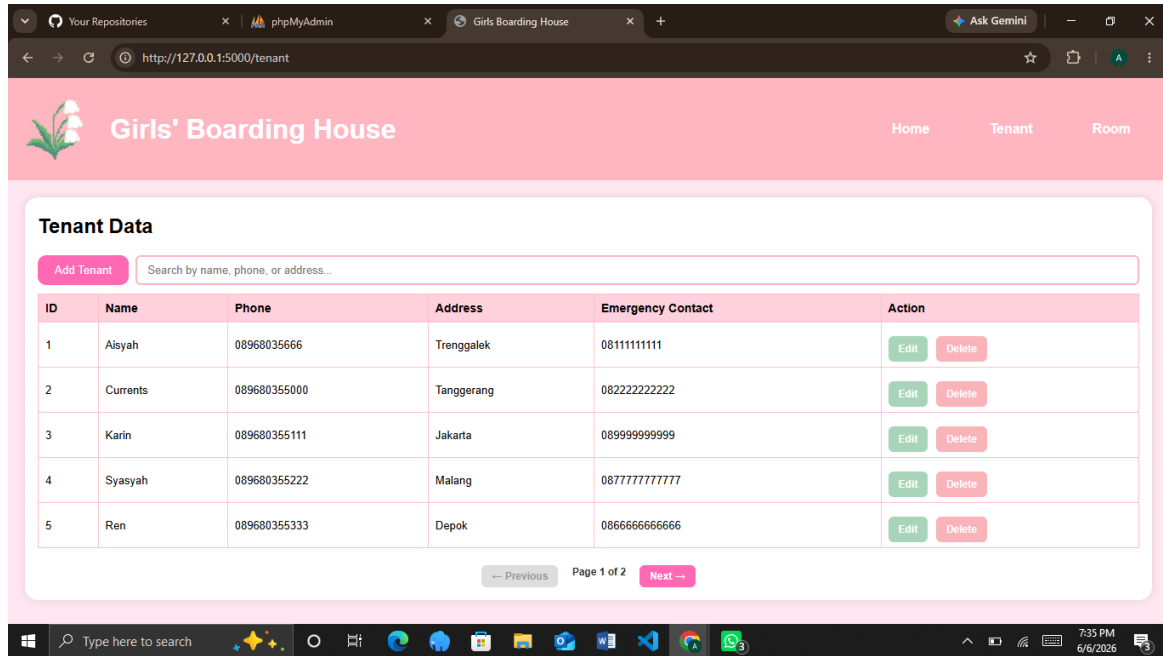


Figure 29. Home page Python Application

The figure above shows the homepage interface of the Girls' Boarding House Information System developed using Python and Flask. The homepage provides a simple and user-friendly interface with a pastel pink theme design to create a comfortable appearance for users. On the navigation bar, there are several menus such as Home, Tenant, and Room, which are used to access different features within the system. The main content section displays a welcome message along with an illustration image to make the interface more attractive and interactive. In addition, the homepage contains a short description explaining that the system is designed to help administrators manage boarding house data efficiently, including tenant and room management. This interface serves as the main page that connects users to all available system features.



ID	Name	Phone	Address	Emergency Contact	Action
1	Aisyah	08968035666	Trenggalek	08111111111	Edit Delete
2	Currents	089680355000	Tangerang	08222222222	Edit Delete
3	Karin	089680355111	Jakarta	08999999999	Edit Delete
4	Syasyah	089680355222	Malang	08777777777	Edit Delete
5	Ren	089680355333	Depok	08666666666	Edit Delete

Figure 30. Tenant page Python Application

The Source code

```
1. @app.route('/tenant')
2. def tenant():
3.     cursor.execute("SELECT * FROM m_tenant")
4.     data = cursor.fetchall()
5.     return render_template('tenant.html', tenants=data)
```

This page is used to display tenant data stored in the *m_tenant* table from the database. The source code above explains the process of retrieving tenant data using the Flask framework. In line 1, the `@app.route('/tenant')` function defines the URL route that will open the tenant page. Line 2 defines the `tenant()` function, which will be executed when the route is accessed. In line 3, the SQL query `SELECT * FROM m_tenant` is used to retrieve all tenant data from the database table. The retrieved data is then stored in the data variable using the `fetchall()` method in line 4. Finally, line 5 renders the `tenant.html` template and sends the tenant data to the webpage so it can be displayed to the user. This feature allows administrators to view and manage tenant information more efficiently through the web application interface.

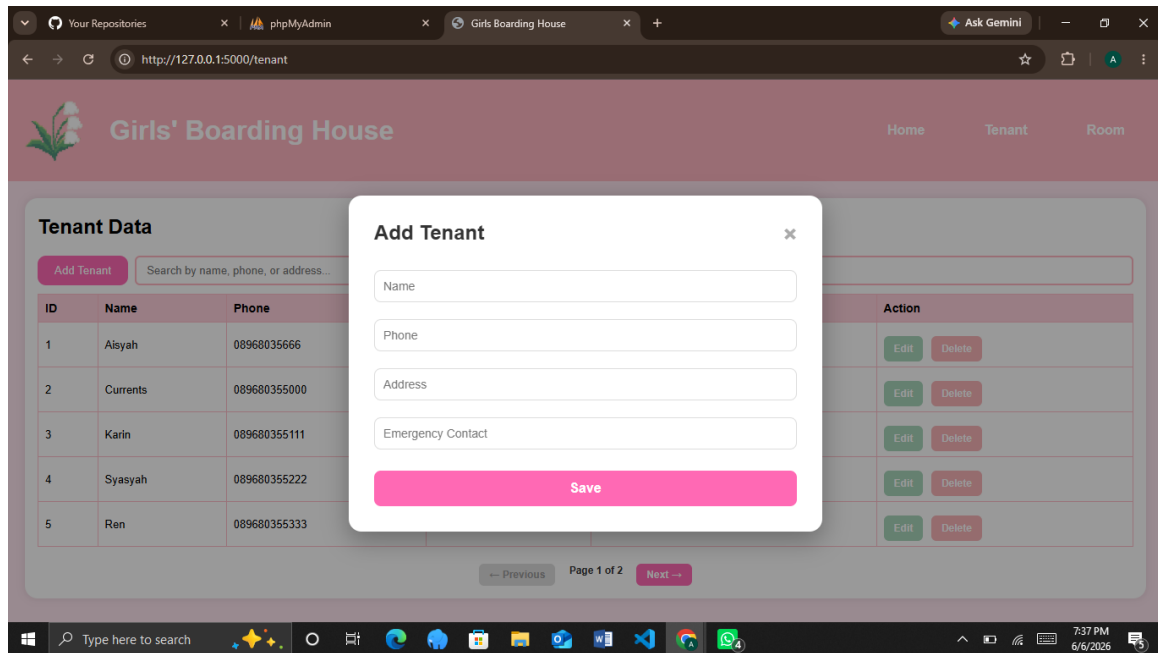


Figure 31. Add tenant Python Application

The Source code

```
1. @app.route('/add_tenant', methods=['GET', 'POST'])
2. def add_tenant():
3.     if request.method == 'POST':
4.         name = request.form['name']
5.         phone = request.form['phone']
6.         address = request.form['address']
7.         emergency_contact = request.form['emergency_contact']
8.
9.         sql = """
10.        INSERT INTO m_tenant(name, phone, address, emergency_contact)
11.        VALUES (%s,%s,%s,%s)
12.        """
13.
14.        values = (name, phone, address, emergency_contact)
15.
16.        cursor.execute(sql, values)
17.        db.commit()
18.
19.        return redirect('/tenant')
20.
21.    return render_template('add_tenant.html')
```

This feature is used to add new tenant data into the `m_tenant` table in the database. The source code above explains how the Flask framework handles the process of adding tenant information through a web form. In line 1, the `@app.route('/add_tenant',`

methods=['GET', 'POST']) function defines the route for the add tenant page and allows both GET and POST methods. Line 2 defines the add_tenant() function that will be executed when the page is accessed. In line 3, the system checks whether the request method is POST, which means the user has submitted the form. Lines 4 to 7 retrieve the input data from the form, including tenant name, phone number, address, and emergency contact. Lines 9 to 12 contain the SQL query used to insert the new tenant data into the *m_tenant* table. The input values are stored in the values variable in line 14 and executed using cursor.execute(sql, values) in line 16. After the data is successfully inserted, line 17 saves the changes to the database using db.commit(). Finally, line 19 redirects the user back to the tenant page, while line 21 displays the add_tenant.html page when the form has not yet been submitted. This feature helps administrators add tenant data quickly and efficiently through the web application.

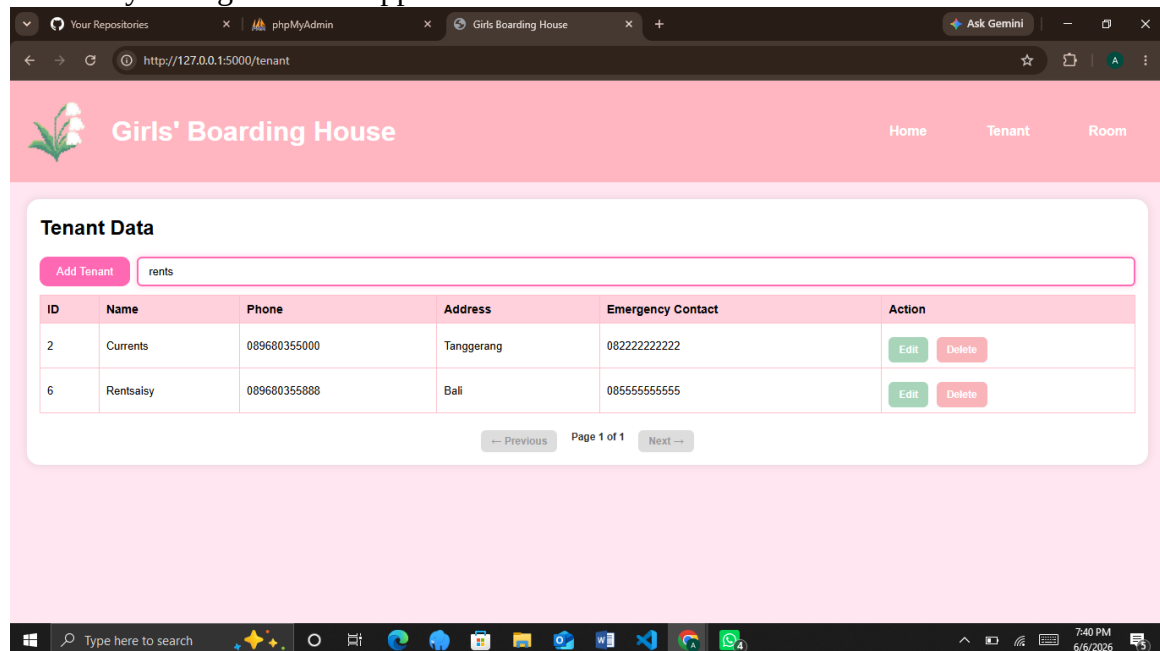


Figure 32. Search tenant Python Application

The search bar is located above the table and allows users to enter keywords such as tenant name, phone number, or address to filter the displayed data. This feature is useful when the amount of tenant data becomes large, as it reduces the time needed to search for specific records manually. Once a keyword is entered, the system automatically displays tenant data that matches the search input. By implementing this search feature, administrators can manage and access tenant information in a faster, more organized, and user-friendly manner.

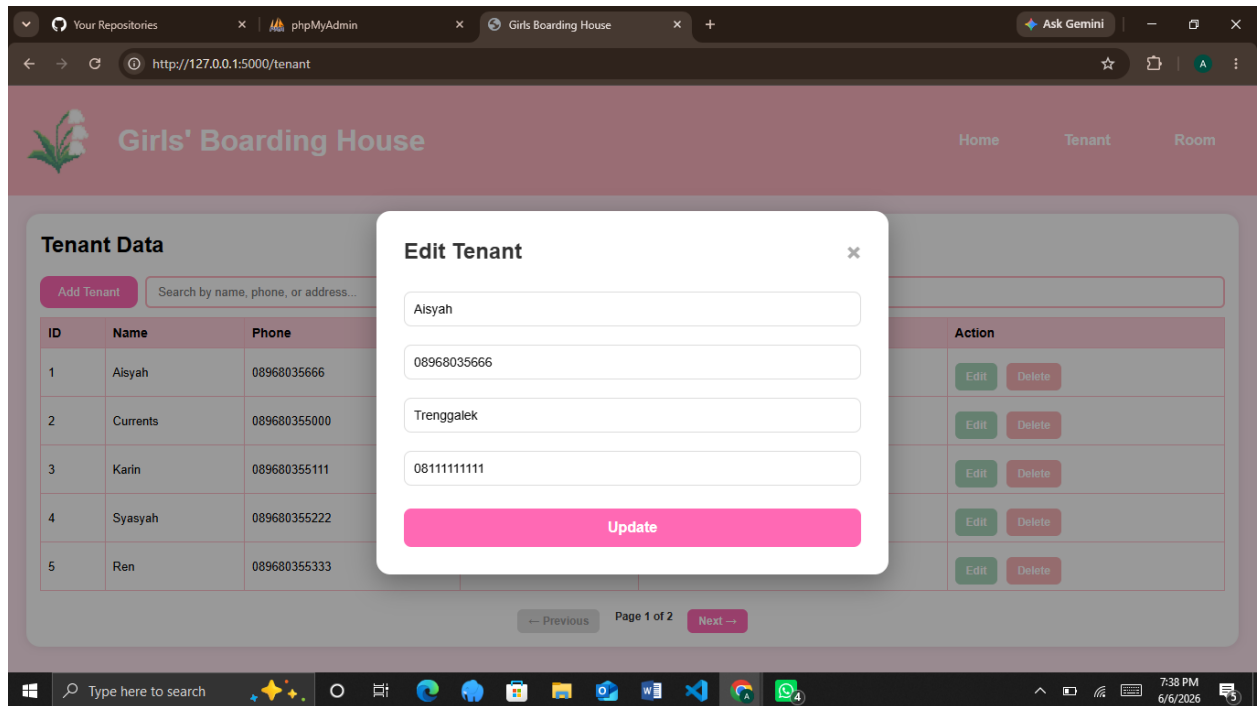


Figure 33. Edit tenant Python Application

The Source code

```
1. @app.route('/edit_tenant/<int:id>', methods=['GET', 'POST'])
2. def edit_tenant(id):
3.     if request.method == 'POST':
4.         name = request.form['name']
5.         phone = request.form['phone']
6.         address = request.form['address']
7.         emergency_contact = request.form['emergency_contact']
8.
9.         sql = """
10.         UPDATE m_tenant SET name=%s, phone=%s, address=%s, emergency_co
            ntact=%s
11.         WHERE tenant_id=%s
12.         """
13.
14.         values = (name, phone, address, emergency_contact, id)
15.         cursor.execute(sql, values)
16.         db.commit()
17.
18.         return redirect('/tenant')
19.
20.     cursor.execute("SELECT * FROM m_tenant WHERE tenant_id=%s", (id,))
21.     tenant = cursor.fetchone()
22.
```

23. `return render_template('edit_tenant.html', tenant=tenant)`

This feature is used to update or modify existing tenant data stored in the *m_tenant* table. The source code above explains how the system processes tenant data editing through a web form. In line 1, the `@app.route('/edit_tenant/<int:id>', methods=['GET', 'POST'])` function defines the route for editing tenant data based on a specific tenant ID and allows both GET and POST methods. Line 2 defines the `edit_tenant(id)` function that will be executed when the page is accessed. In line 3, the system checks whether the form has been submitted using the POST method. Lines 4 to 7 retrieve updated tenant information from the form, including name, phone number, address, and emergency contact. Lines 9 to 12 contain the SQL query used to update the tenant data in the *m_tenant* table based on the selected tenant ID. The updated values are stored in the `values` variable in line 14 and executed using `cursor.execute(sql, values)` in line 15. After the data is successfully updated, line 16 saves the changes to the database using `db.commit()`, and line 18 redirects the user back to the tenant page. Meanwhile, lines 20 and 21 retrieve the existing tenant data from the database using the tenant ID so that the previous information can be displayed in the edit form before changes are made. This feature helps administrators manage tenant data more accurately and efficiently within the system.

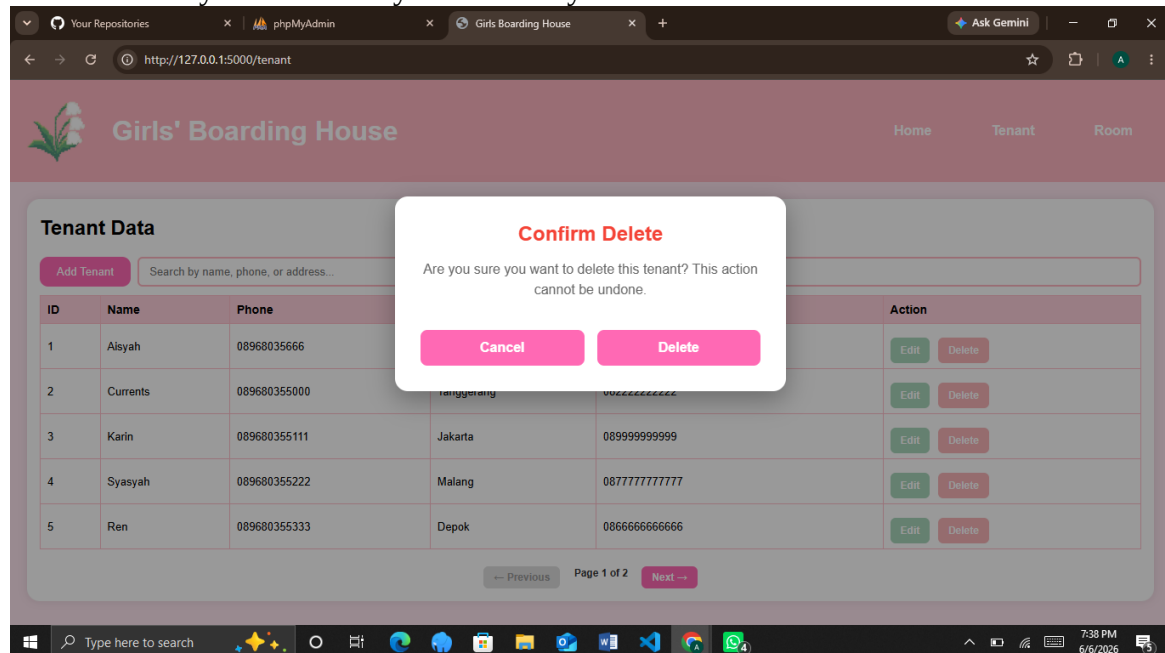


Figure 34. Delete tenant Python Application

The Source code

```
1. @app.route('/delete_tenant/<int:id>')
2. def delete_tenant(id):
3.     cursor.execute("DELETE FROM m_tenant WHERE tenant_id=%s", (id,))
4.     db.commit()
5.
6.     return redirect('/tenant')
```

This feature is used to remove tenant data from the *m_tenant* table in the database. The source code above explains the process of deleting tenant information based on a specific tenant ID. In line 1, the `@app.route('/delete_tenant/<int:id>')` function defines the route for deleting tenant data and receives the tenant ID as a parameter. Line 2 defines the `delete_tenant(id)` function that will be executed when the route is accessed. In line 3, the SQL query `DELETE FROM m_tenant WHERE tenant_id=%s` is used to remove the selected tenant data from the database based on the provided ID. The `(id,)` parameter ensures that the correct tenant record is deleted securely. After the deletion process is completed, line 4 saves the changes to the database using `db.commit()`. Finally, line 6 redirects the user back to the tenant page so the updated tenant data can be displayed immediately. This feature helps administrators manage tenant records efficiently by removing unnecessary or inactive tenant data from the system.

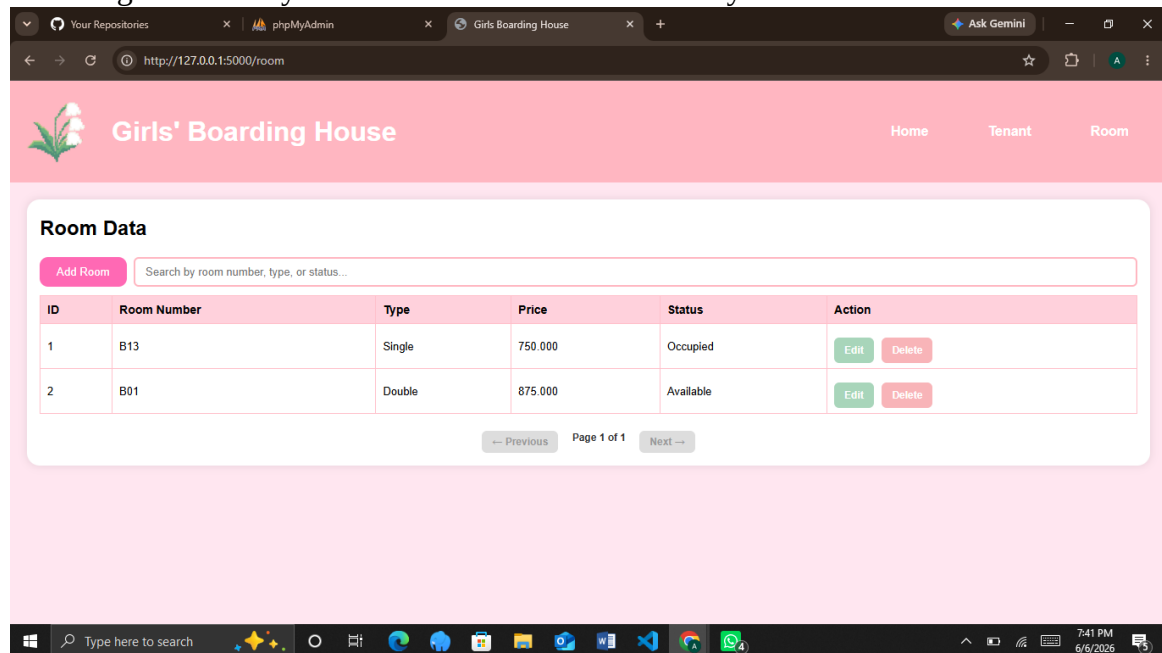


Figure 35. Room page Python Application

The Source code

```
1. @app.route('/room')
2. def room():
3.     cursor.execute("SELECT * FROM m_room")
4.     data = cursor.fetchall()
5.
6.     return render_template('room.html', rooms=data)
```

This page is used to display room data stored in the *m_room* table within the database. The source code above explains the process of retrieving and displaying room information through the web application. In line 1, the `@app.route('/room')` function defines the route used to access the room page. Line 2 defines the `room()` function that will be executed when the route is opened by the user. In line 3, the SQL query `SELECT *`

FROM `m_room` is used to retrieve all room data from the database table. The retrieved data is then stored in the data variable using the `fetchall()` method in line 4. Finally, line 6 renders the `room.html` template and sends the room data to the webpage so it can be displayed in a table format for users. This feature helps administrators view and manage room information more efficiently, including room availability and other related details within the boarding house system.

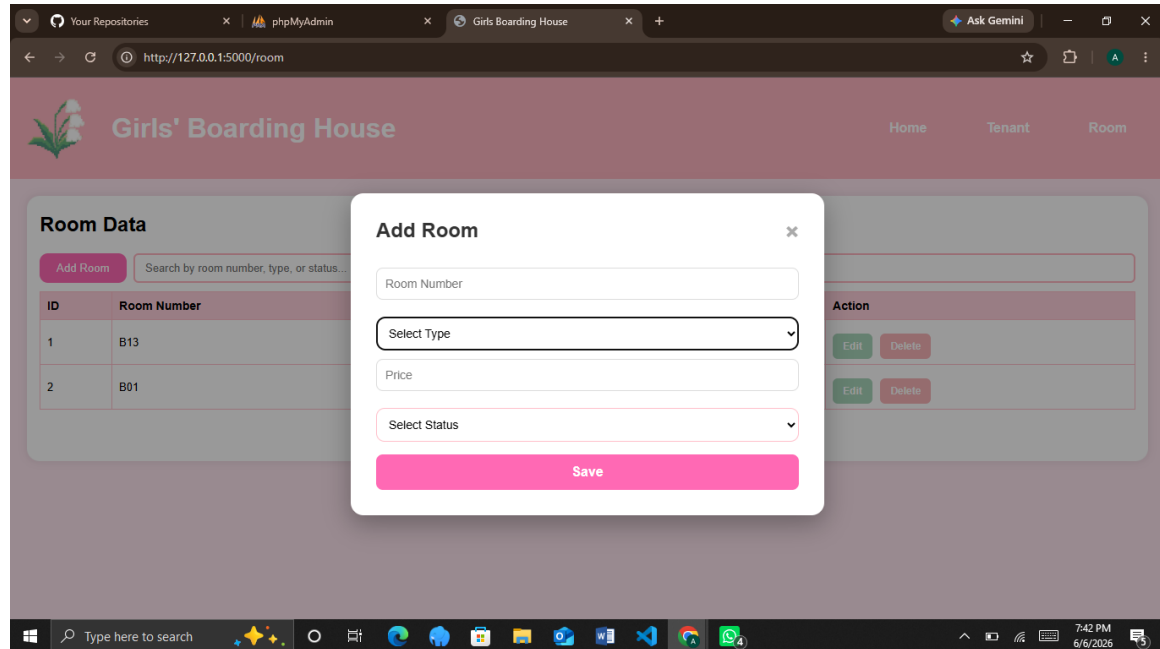


Figure 36. Add room Python Application

The Source code

```
1. @app.route('/add_room', methods=['GET', 'POST'])
2. def add_room():
3.     if request.method == 'POST':
4.         room_number = request.form['room_number']
5.         type = request.form['type']
6.         price = request.form['price']
7.         status = request.form['status']
8.         sql = """
9.         INSERT INTO m_room(room_number, type, price, status)
10.        VALUES (%s,%s,%s,%s)
11.        """
12.        values = (room_number, type, price, status)
13.        cursor.execute(sql, values)
14.        db.commit()
15.        return redirect('/room')
16.    return render_template('add_room.html')
```

This feature is used to add new room data into the `m_room` table in the database. The source code above explains the process of handling room data input through a web

form. In line 1, the `@app.route('/add_room', methods=['GET', 'POST'])` function defines the route for the add room page and allows both GET and POST methods. Line 2 defines the `add_room()` function that will be executed when the route is accessed. In line 3, the system checks whether the form has been submitted using the POST method. Lines 4 to 7 retrieve the room information entered by the user, including room number, room type, price, and room status. Lines 8 to 11 contain the SQL query used to insert the new room data into the `m_room` table. The input values are stored in the `values` variable in line 12 and executed using `cursor.execute(sql, values)` in line 13. After the data is successfully inserted into the database, line 14 saves the changes using `db.commit()`. Line 15 then redirects the user back to the room page so the newly added room data can be displayed immediately. Meanwhile, line 16 renders the `add_room.html` page when the form has not yet been submitted. This feature helps administrators manage room information more efficiently and systematically within the boarding house management system.

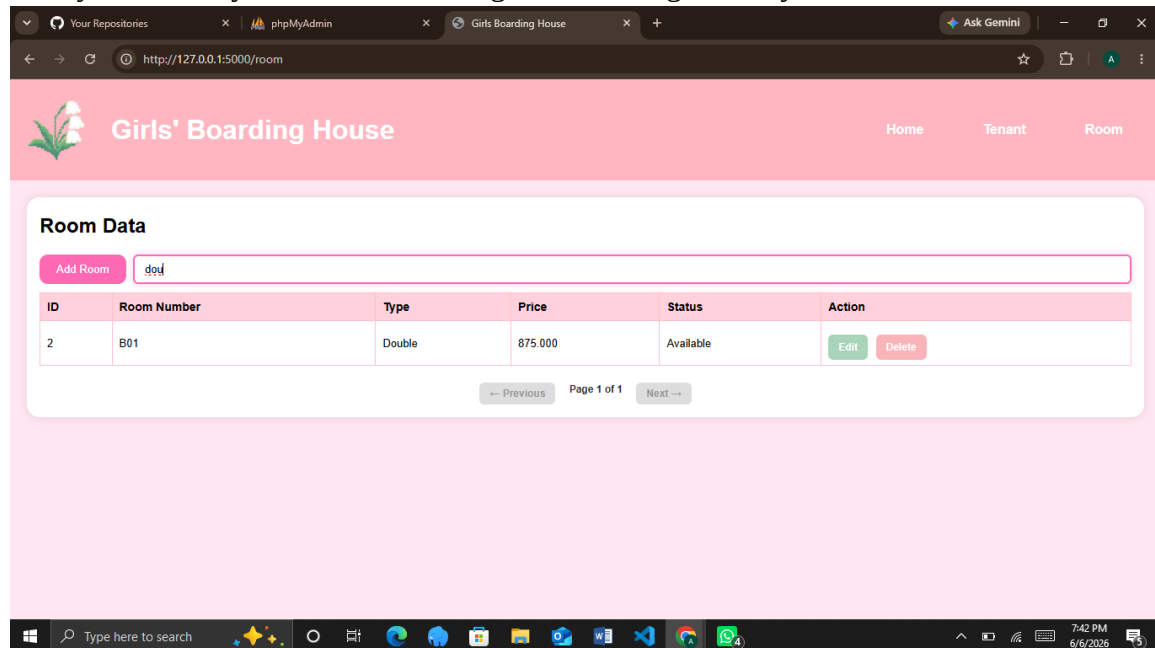


Figure 37. Search room Python Application

This feature is used to help administrators search and filter room data quickly and efficiently. The search bar is located above the room data table and allows users to enter keywords related to room information, such as room number, room type, price, or room status. After a keyword is entered, the system automatically displays room data that matches the search input, making it easier for administrators to find specific room information without manually checking all records. In the figure above, the keyword entered in the search field filters the displayed room data based on the matching result. This feature improves the efficiency of room data management and provides a more user-friendly experience within the boarding house information system.

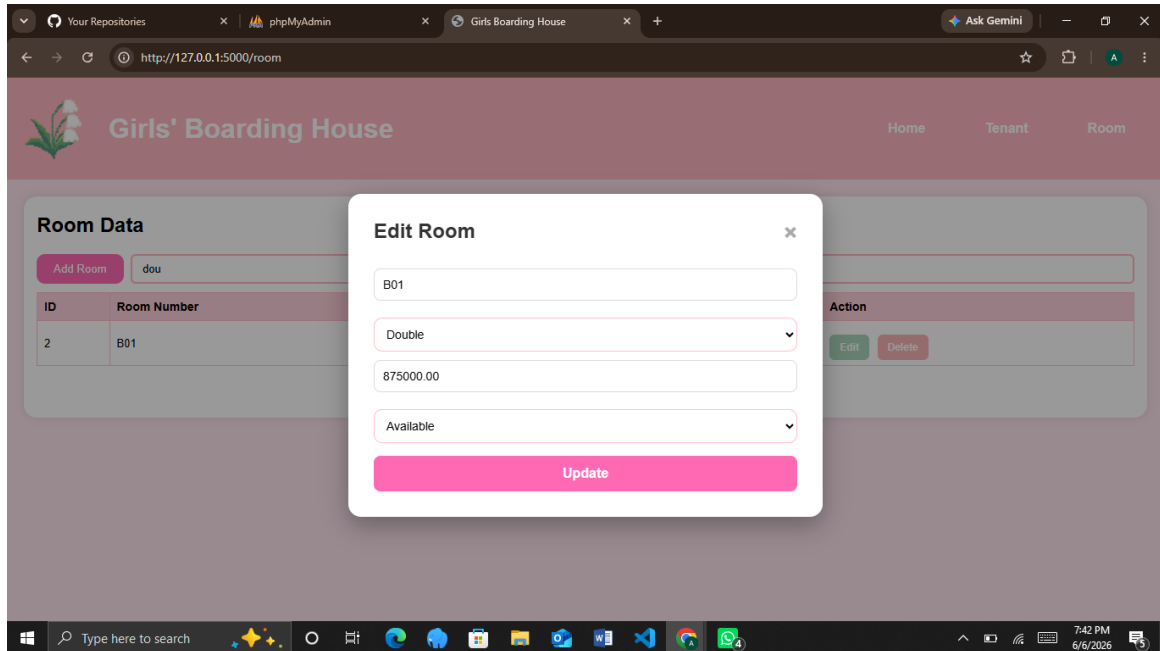


Figure 38. Edit room Python Application

The Source code

```
1. @app.route('/edit_room/<int:id>', methods=['GET', 'POST'])
2. def edit_room(id):
3.
4.     if request.method == 'POST':
5.         room_number = request.form['room_number']
6.         type = request.form['type']
7.         price = request.form['price']
8.         status = request.form['status']
9.
10.        sql = """
11.        UPDATE m_room
12.        SET room_number=%s, type=%s, price=%s, status=%s
13.        WHERE room_id=%s
14.        """
15.
16.        values = (room_number, type, price, status, id)
17.
18.        cursor.execute(sql, values)
19.        db.commit()
20.
21.        return redirect('/room')
22.
23.        cursor.execute("SELECT * FROM m_room WHERE room_id=%s", (id,))
24.        room = cursor.fetchone()
```

25.

26. `return render_template('edit_room.html', room=room)`

This feature is used to update or modify existing room data stored in the *m_room* table within the database. The source code above explains the process of editing room information through a web form. In line 1, the `@app.route('/edit_room/<int:id>', methods=['GET', 'POST'])` function defines the route for editing room data based on a specific room ID and allows both GET and POST methods. Line 2 defines the `edit_room(id)` function that will be executed when the route is accessed. In line 4, the system checks whether the form has been submitted using the POST method. Lines 5 to 8 retrieve updated room information entered by the user, including room number, room type, price, and room status. Lines 10 to 14 contain the SQL query used to update room data in the *m_room* table based on the selected room ID. The updated values are stored in the `values` variable in line 16 and executed using `cursor.execute(sql, values)` in line 18. After the data is successfully updated, line 19 saves the changes to the database using `db.commit()`, while line 21 redirects the user back to the room page to display the updated information. Meanwhile, lines 23 and 24 retrieve the existing room data from the database using the room ID so the previous information can be displayed in the edit form before modifications are made. Finally, line 26 renders the `edit_room.html` template and sends the room data to the webpage. This feature helps administrators manage room information more accurately, efficiently, and systematically within the boarding house management system.

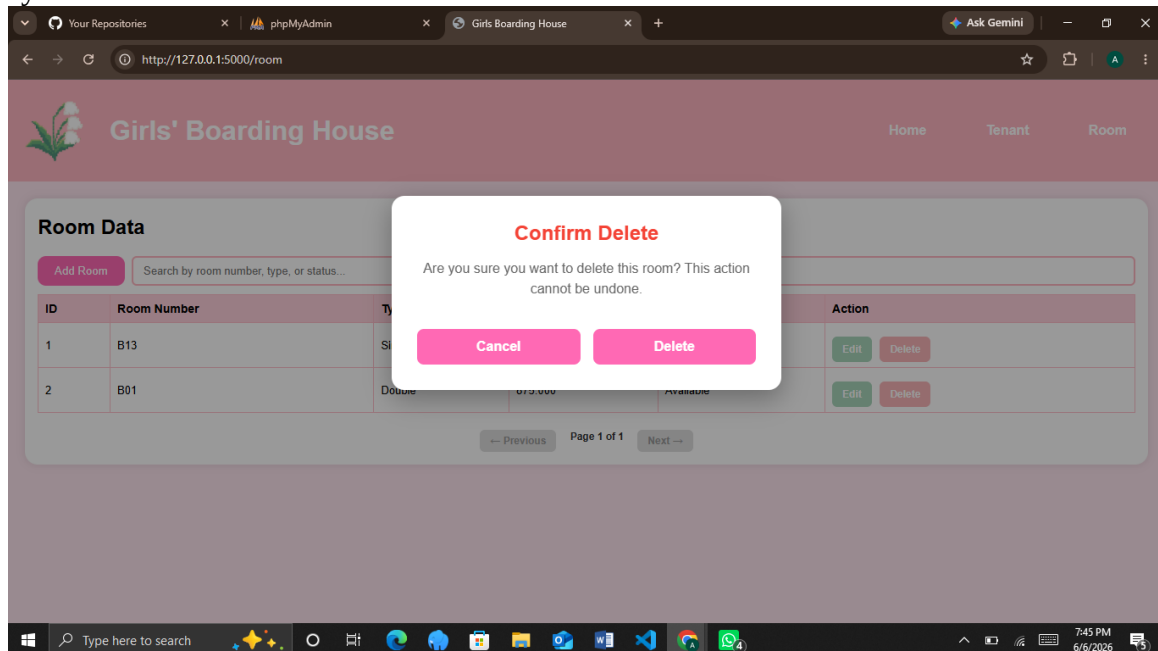


Figure 39. Delete room Python Application

The Source code

```
1. @app.route('/delete_room/<int:id>')
2. def delete_room(id):
3.
```

```
4. cursor.execute("DELETE FROM m_room WHERE room_id=%s", (id,))
5. db.commit()
6.
7. return redirect('/room')
```

This feature is used to remove room data from the *m_room* table in the database. The source code above explains the process of deleting room information based on a specific room ID. In line 1, the `@app.route('/delete_room/<int:id>')` function defines the route used to access the delete room feature and receives the room ID as a parameter. Line 2 defines the `delete_room(id)` function that will be executed when the route is accessed. In line 4, the SQL query `DELETE FROM m_room WHERE room_id=%s` is used to remove the selected room data from the database based on the provided room ID. The `(id,)` parameter ensures that the correct room record is deleted securely. After the deletion process is completed, line 5 saves the changes to the database using `db.commit()`. Finally, line 7 redirects the user back to the room page so the updated room data can be displayed immediately. This feature helps administrators manage room records more efficiently by removing unused or unnecessary room data from the boarding house management system.

2. PHP Application

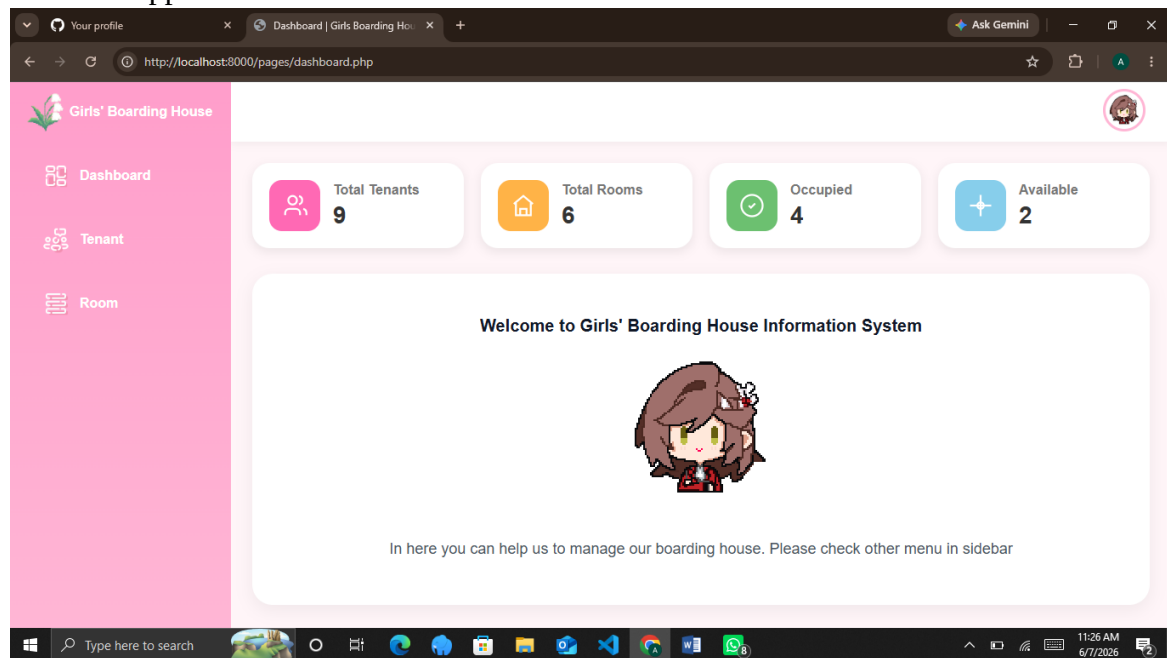


Figure 40. Dashboard PHP Application

This dashboard serves as the main interface that provides an overview of important boarding house information for administrators. The page displays several statistical cards, including the total number of tenants, total rooms, occupied rooms, and available rooms, allowing users to monitor boarding house conditions quickly and efficiently. In addition, the dashboard features a sidebar navigation menu consisting of Dashboard, Tenant, and Room pages to help users access different system functions easily. The main content

section also includes a welcome message and an illustration image to create a more attractive and user-friendly interface. Overall, the dashboard page helps administrators manage boarding house data in a more organized, efficient, and visually appealing manner.

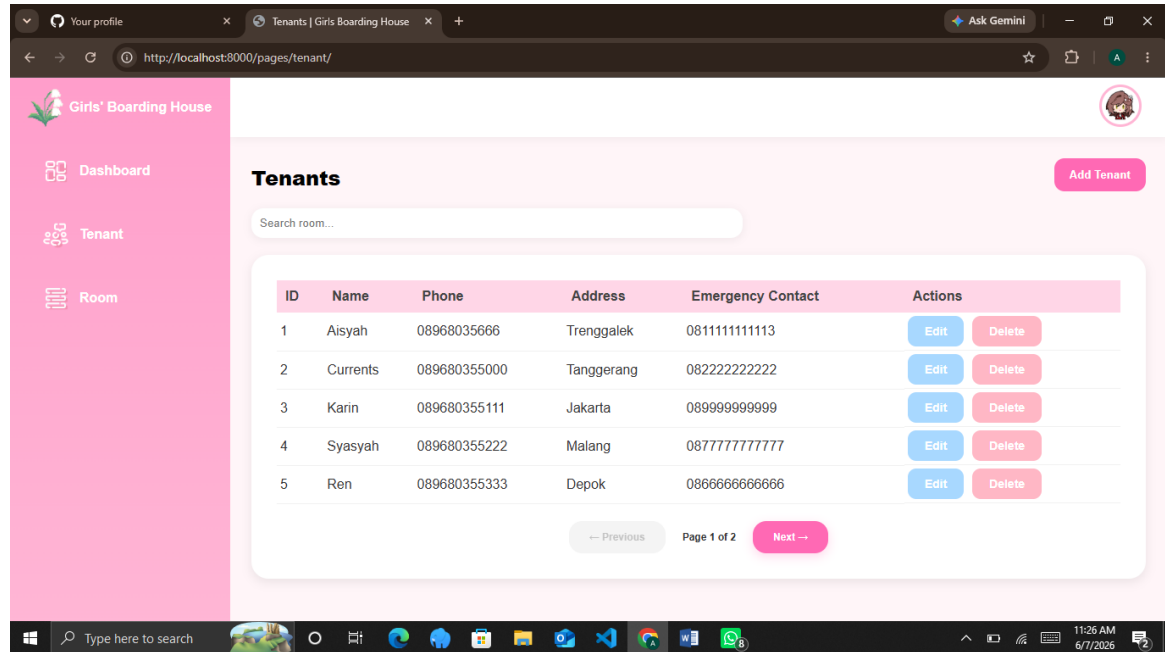


Figure 41. Tenant pages PHP Application

The Source code

```
1. $result = $conn->query("
2.     SELECT *
3.     FROM m_tenant
4.     ORDER BY tenant_id ASC
5.     LIMIT $start, $limit
6. ");
```

This page is used to display tenant data stored in the *m_tenant* table within the database. The source code above explains the process of retrieving tenant information using an SQL query in PHP. In lines 1 to 5, the `$conn->query()` function executes an SQL query to select all tenant data from the *m_tenant* table. The `ORDER BY tenant_id ASC` statement is used to sort the tenant data in ascending order based on the tenant ID, making the displayed information more organized and easier to read. Meanwhile, the `LIMIT $start, $limit` statement is implemented to apply pagination, allowing the system to display data in smaller sections instead of showing all records at once. This helps improve page performance and makes navigation easier when the amount of tenant data becomes large. Overall, this feature helps administrators manage and view tenant information more efficiently through the PHP-based web application.

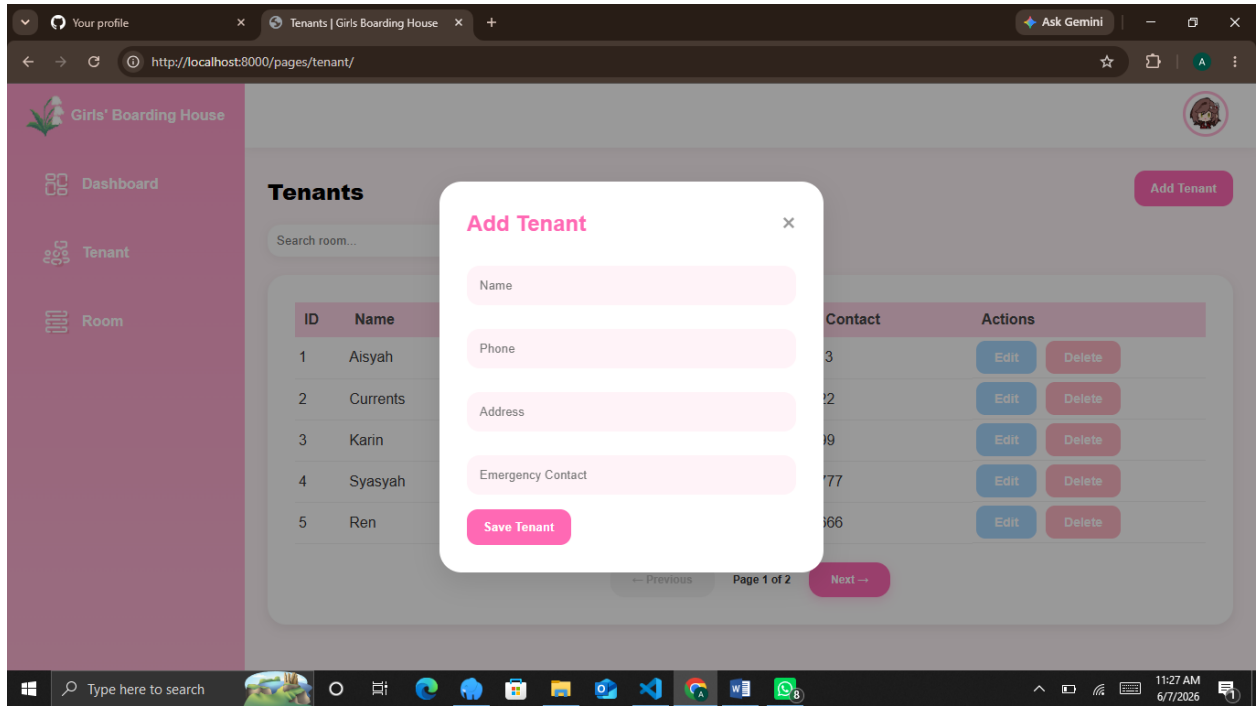


Figure 42. Add tenant PHP Application

The Source code

```
1. if($_SERVER['REQUEST_METHOD'] == 'POST' && !isset($_POST['update_tenant
   '])){
2.
3.     $name =
4.     $_POST['name'];
5.
6.     $phone =
7.     $_POST['phone'];
8.
9.     $address =
10.    $_POST['address'];
11.
12.    $emergency_contact =
13.    $_POST['emergency_contact'];
14.
15.    $stmt = $conn->prepare("
16.        INSERT INTO m_tenant
17.        (
18.            name,
19.            phone,
20.            address,
21.            emergency_contact
22.        )
```

```
23.         VALUES (?, ?, ?, ?)
24.     ");
25.
26.     $stmt->bind_param(
27.         "ssss",
28.         $name,
29.         $phone,
30.         $address,
31.         $emergency_contact
32.     );
33.
34.     $stmt->execute();
35.
36.     header("Location: index.php");
37.     exit;
38. }
```

This feature is used to add new tenant data into the *m_tenant* table in the database. The source code above explains the process of handling tenant data input through a web form using PHP and MySQL. In line 1, the system checks whether the request method is POST and ensures that the submitted form is intended for adding tenant data instead of updating existing data. Lines 3 to 13 retrieve the input values entered by the user, including tenant name, phone number, address, and emergency contact information from the form fields using the `$_POST` method.

In lines 15 to 24, the `$conn->prepare()` function is used to create a prepared SQL statement for inserting new tenant data into the *m_tenant* table. The prepared statement helps improve security by preventing SQL injection attacks. Lines 26 to 32 use the `bind_param()` function to bind the user input values into the SQL query with the "ssss" parameter type, which indicates that all inserted values are strings. After the data is successfully prepared, line 34 executes the query using `$stmt->execute()` to save the new tenant data into the database. Finally, lines 36 and 37 redirect the user back to the main page after the insertion process is completed successfully. This feature helps administrators add tenant information more efficiently, securely, and systematically within the PHP-based boarding house management system.

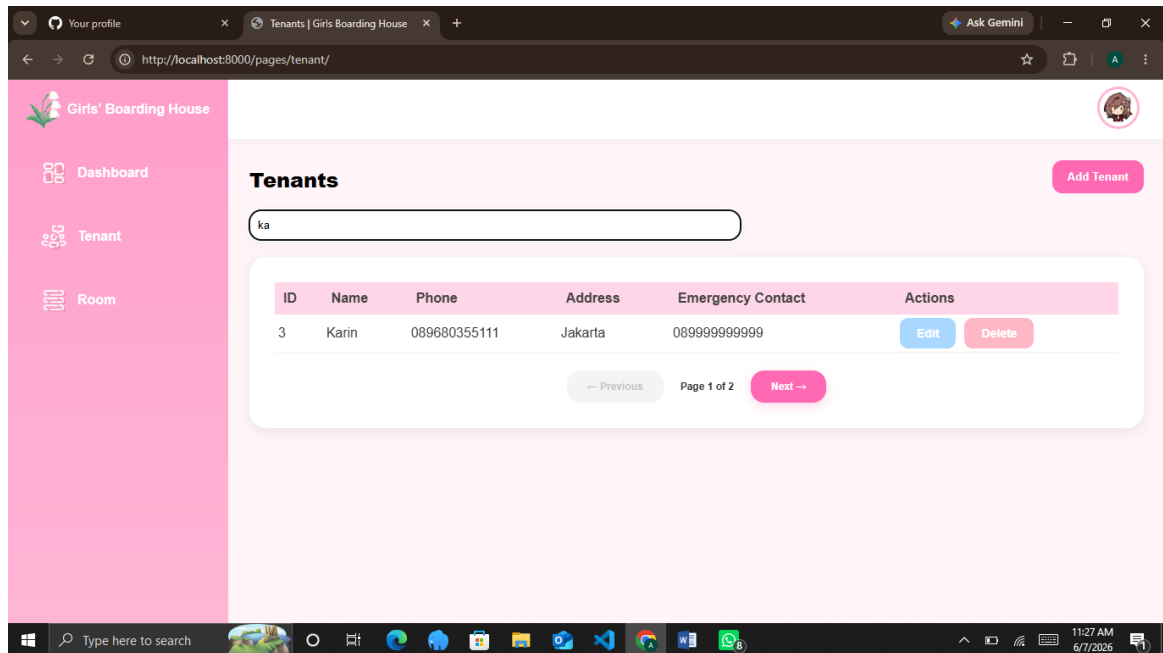


Figure 43. Search tenant PHP Application

This feature allows administrators to search tenant data quickly by entering keywords into the search bar. The system automatically filters and displays tenant information that matches the entered keyword, such as tenant name, phone number, or address. In the figure above, the keyword entered in the search field helps narrow down the displayed data, making it easier for administrators to find specific tenant records without checking all available data manually. This feature improves efficiency and provides a more user-friendly experience in managing tenant information.

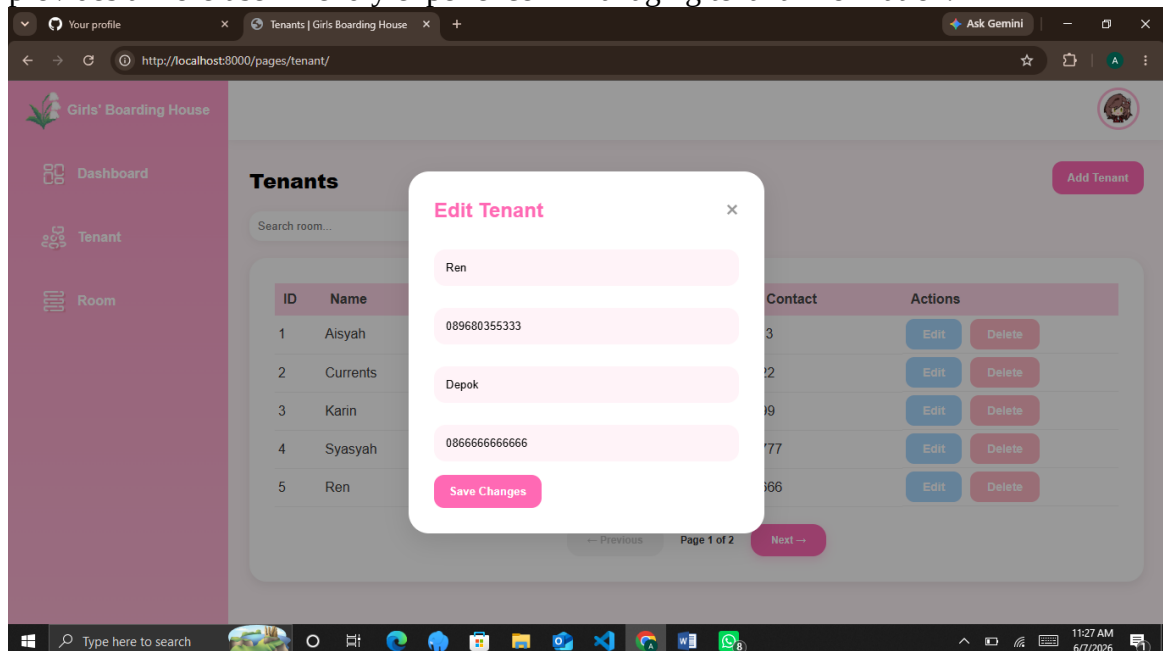


Figure 44. Edit tenant PHP Application

The Source code

```
1. if(isset($_POST['update_tenant'])){
2.
3.     $id = $_POST['tenant_id'];
4.
5.     $name = $_POST['name'];
6.
7.     $phone = $_POST['phone'];
8.
9.     $address = $_POST['address'];
10.
11.    $emergency = $_POST['emergency_contact'];
12.
13.    $stmt = $conn->prepare("
14.        UPDATE m_tenant
15.        SET
16.            name=?,
17.            phone=?,
18.            address=?,
19.            emergency_contact=?
20.        WHERE tenant_id=?
21.    ");
22.
23.    $stmt->bind_param(
24.        "ssssi",
25.        $name,
26.        $phone,
27.        $address,
28.        $emergency,
29.        $id
30.    );
31.
32.    $stmt->execute();
33.
34.    header("Location:index.php");
35.    exit;
36.}
```

This feature is used to update existing tenant data stored in the *m_tenant* table. The source code checks whether the update form has been submitted using `isset($_POST['update_tenant'])`, then retrieves tenant information such as tenant ID, name, phone number, address, and emergency contact from the form input. After that, the system uses a prepared SQL statement to update the selected tenant data securely in the database based on the tenant ID. The `bind_param()` function is used to bind the input values into the

SQL query before executing it. Once the update process is completed successfully, the system redirects the user back to the main tenant page. This feature helps administrators modify tenant information efficiently, accurately, and securely.

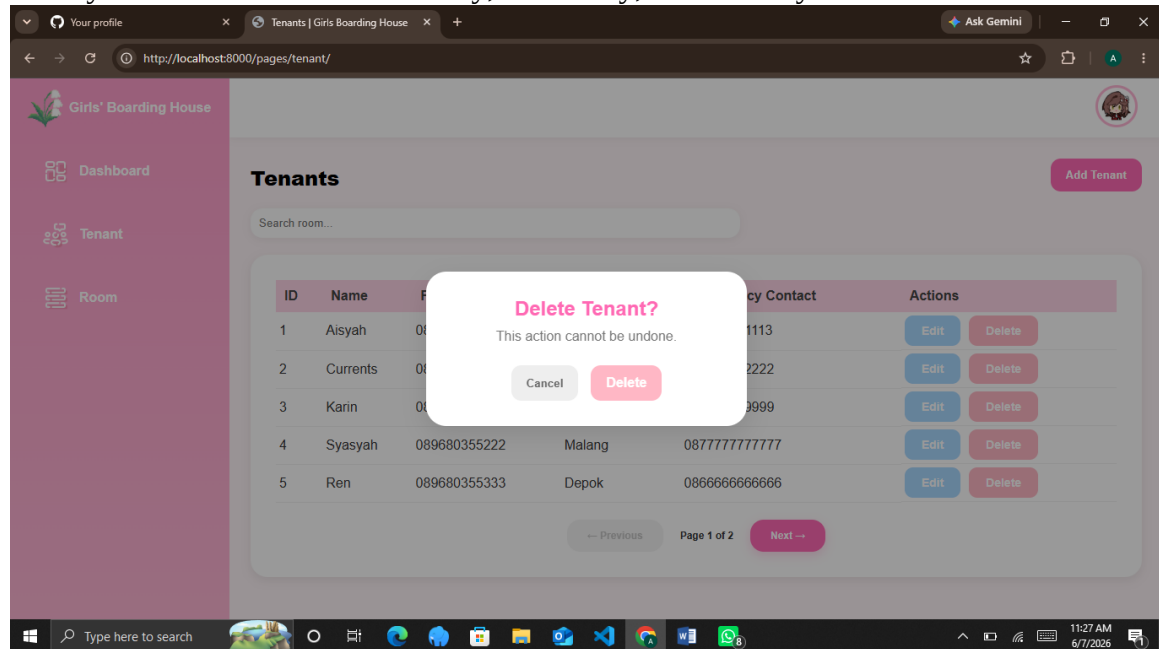


Figure 45. Delete tenant PHP Application

The Source code

```
1. if(isset($_GET['delete'])) {
2.     $id = intval($_GET['delete']);
3.     $conn->query("DELETE FROM m_tenant WHERE tenant_id = $id");
4.     header('Location: index.php');
5.     exit;
6. }
```

This feature is used to remove tenant data from the *m_tenant* table in the database. The source code checks whether the delete parameter is received using `isset($_GET['delete'])`. After that, the tenant ID is retrieved and converted into an integer using `intval()` for safer data processing. The system then executes the SQL query `DELETE FROM m_tenant WHERE tenant_id = $id` to remove the selected tenant record from the database. Once the deletion process is completed successfully, the system redirects the user back to the main tenant page using `header('Location: index.php')`. This feature helps administrators manage tenant records more efficiently by deleting unnecessary or inactive tenant data from the system.

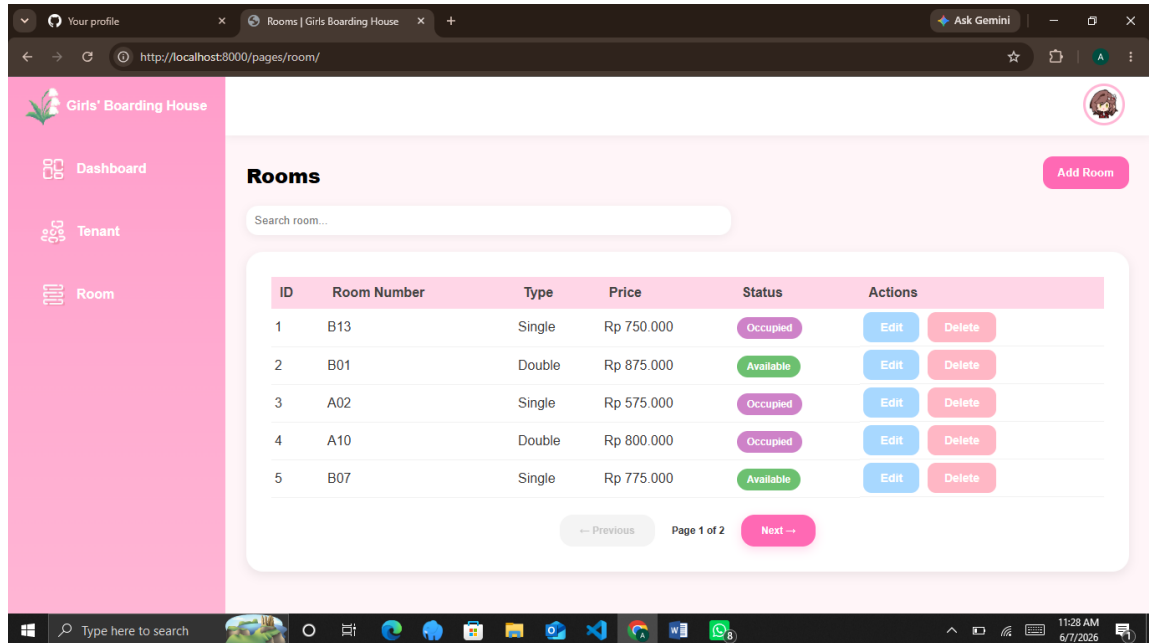


Figure 46. Room pages PHP Application

The Source code

```
1. $result = $conn->query("
2. SELECT *
3. FROM m_room
4. ORDER BY room_id ASC
5. LIMIT $start, $limit
6. ");
```

This feature is used to display room data stored in the *m_room* table within the database. The source code above executes an SQL query to retrieve all room information and sort the data in ascending order based on the room ID using *ORDER BY room_id ASC*. In addition, the *LIMIT \$start, \$limit* statement is implemented to apply pagination, allowing the system to display room data in smaller sections instead of showing all records at once. This helps improve system performance and makes room data easier to manage and navigate when the amount of data becomes large. Overall, this feature helps administrators view and manage room information more efficiently through the PHP-based web application.

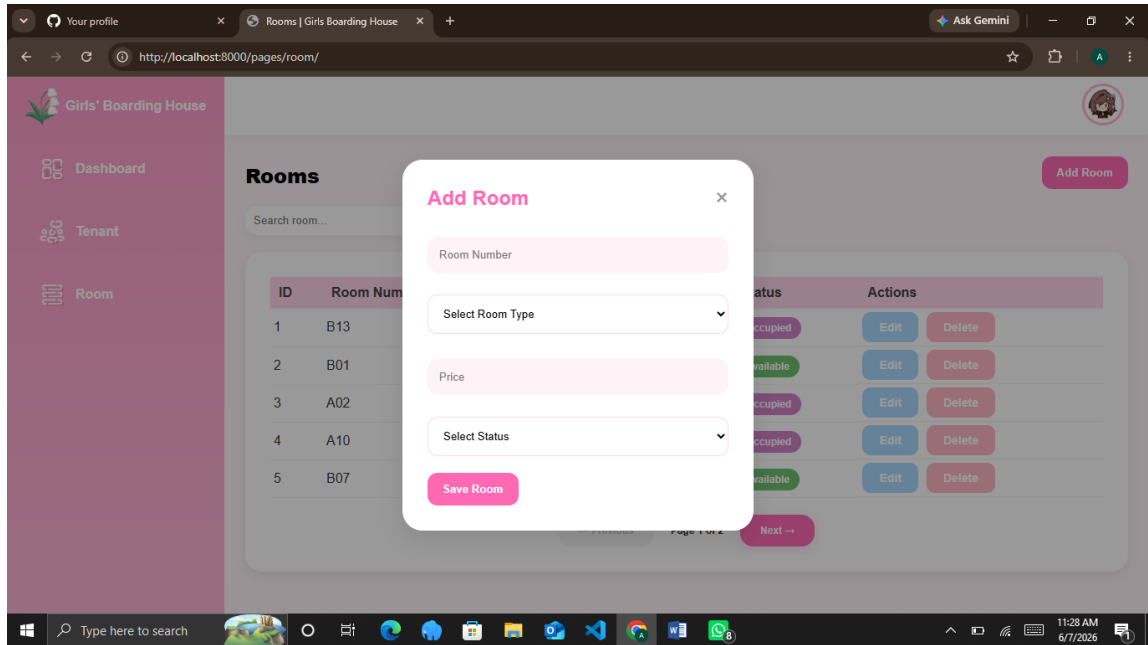


Figure 47. Add room PHP Application

The Source code

```

1. if(
2.     $_SERVER['REQUEST_METHOD'] == 'POST'
3.     &&
4.     !isset($_POST['update_room'])
5. ){
6.
7.     $room_number =
8.     $_POST['room_number'];
9.
10.    $type =
11.    $_POST['type'];
12.
13.    $price =
14.    $_POST['price'];
15.
16.    $status =
17.    $_POST['status'];
18.
19.    $stmt = $conn->prepare("
20.        INSERT INTO m_room
21.        (
22.            room_number,
23.            type,
24.            price,

```

```

25.         status
26.     )
27.     VALUES (?, ?, ?, ?)
28. ");
29.
30.     $stmt->bind_param(
31.         "ssis",
32.         $room_number,
33.         $type,
34.         $price,
35.         $status
36.     );
37.     $stmt->execute();
38.     header("Location: index.php");
39.     exit;
40. }

```

This feature is used to insert new room data into the *m_room* table in the database. The source code checks whether the form is submitted using the POST method and ensures that the process is not for updating existing room data. After that, the system retrieves room information such as room number, type, price, and status from the form input. A prepared SQL statement is then used to securely insert the data into the database and prevent SQL injection attacks. The `bind_param()` function binds the input values into the SQL query before execution. Once the data is successfully stored, the system redirects the user back to the main room page. This feature helps administrators add room information efficiently, accurately, and securely within the PHP-based boarding house management system.

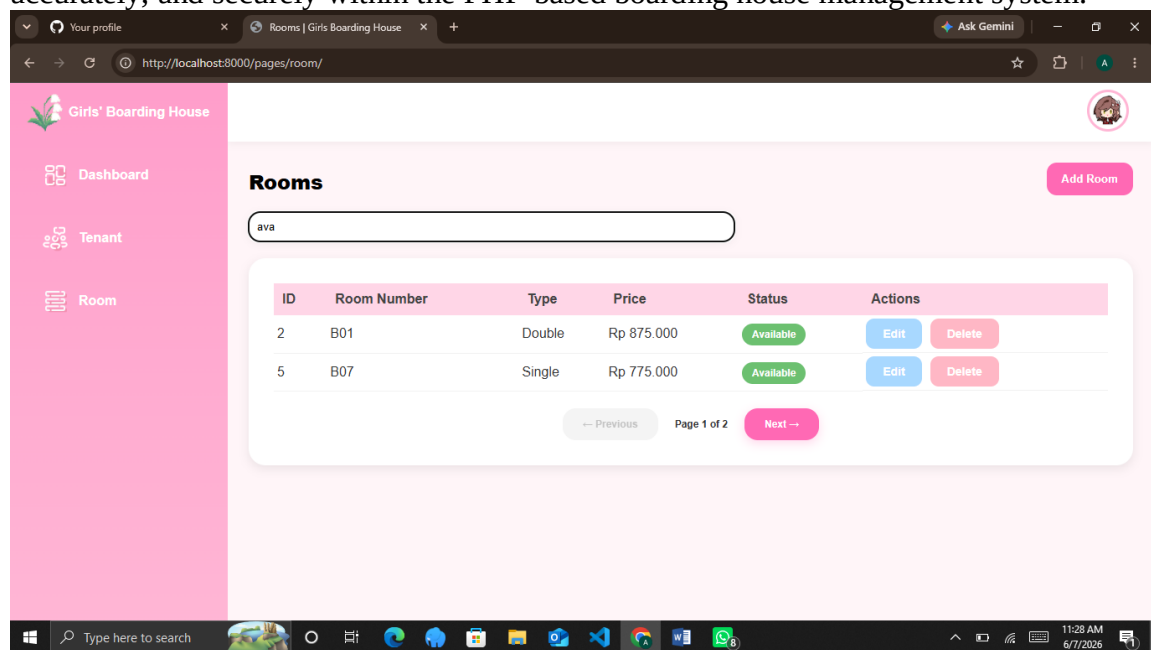


Figure 48. Search room PHP Application

This feature allows administrators to search room data quickly and efficiently by entering keywords into the search bar. The system automatically filters and displays room information that matches the entered keyword, such as room number, room type, price, or room status. In the figure above, the keyword entered in the search field helps narrow down the displayed room data, making it easier for administrators to find specific room records without manually checking all available data. This feature improves efficiency and provides a more user-friendly experience in managing room information within the boarding house management system.

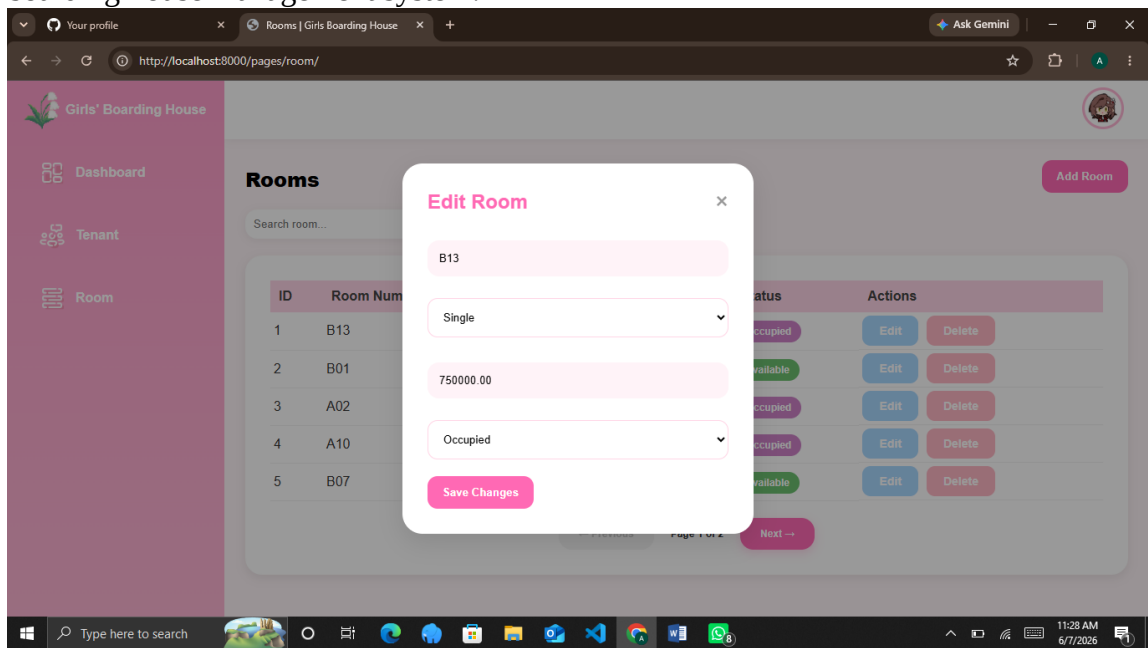


Figure 49. Edit room PHP Application

The Source code

```
1. if(isset($_POST['update_room'])){
2.
3.     $id =
4.     $_POST['room_id'];
5.
6.     $room_number =
7.     $_POST['room_number'];
8.
9.     $type =
10.    $_POST['type'];
11.
12.    $price =
13.    $_POST['price'];
14.
15.    $status =
16.    $_POST['status'];
```

```
17.  
18.     $stmt = $conn->prepare("  
19.         UPDATE m_room  
20.         SET  
21.             room_number=?,  
22.             type=?,  
23.             price=?,  
24.             status=?  
25.         WHERE room_id=?  
26.     ");  
27.  
28.     $stmt->bind_param(  
29.         "ssisi",  
30.         $room_number,  
31.         $type,  
32.         $price,  
33.         $status,  
34.         $id  
35.     );  
36.  
37.     $stmt->execute();  
38.  
39.     header("Location: index.php");  
40.     exit;  
41. }
```

This feature is used to update existing room data stored in the *m_room* table within the database. The source code checks whether the update form has been submitted using `isset($_POST['update_room'])`, then retrieves room information such as room ID, room number, type, price, and status from the form input. After that, the system uses a prepared SQL statement to securely update the selected room data in the database based on the room ID. The `bind_param()` function is used to bind the input values into the SQL query before executing it. Once the update process is completed successfully, the system redirects the user back to the main room page. This feature helps administrators modify room information efficiently, accurately, and securely within the PHP-based boarding house management system.

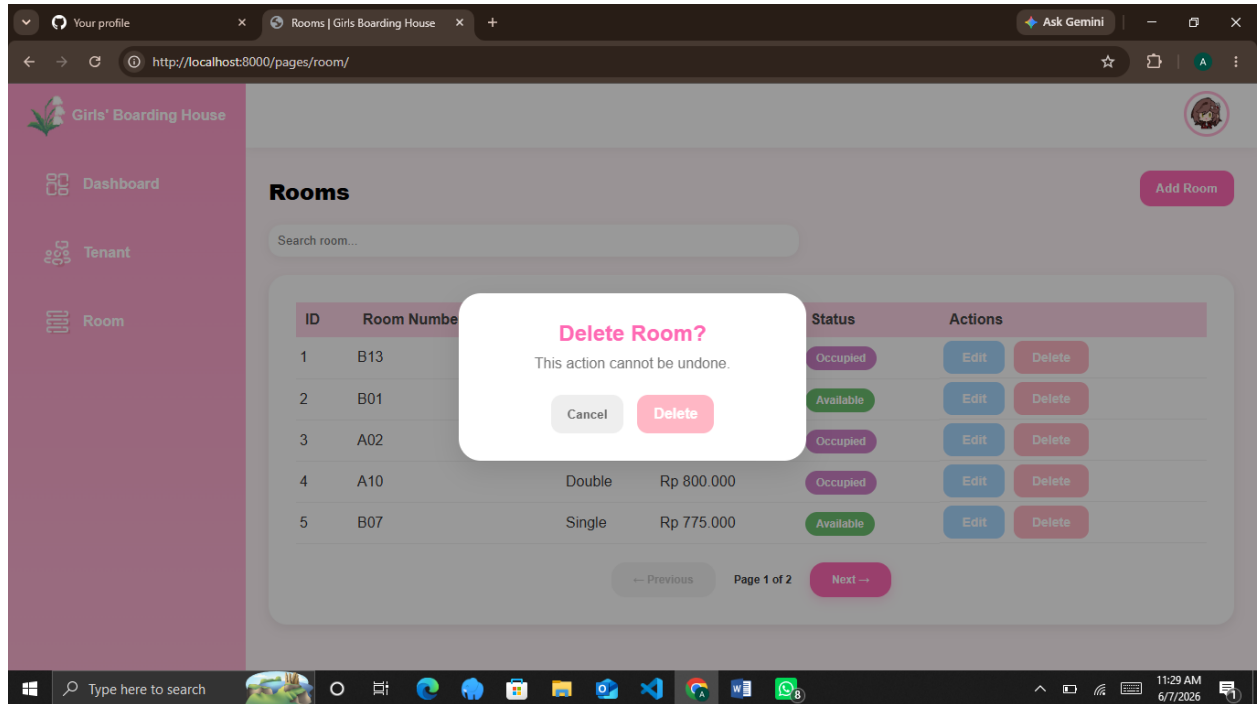


Figure 50. Delete room PHP Application

The Source code

```
1. if(isset($_GET['delete'])) {
2.
3.     $id = intval($_GET['delete']);
4.
5.     $conn->query("
6.         DELETE FROM m_room
7.         WHERE room_id = $id
8.     ");
9.
10.    header('Location: index.php');
11.    exit;
12.}
```

This feature is used to remove room data from the *m_room* table in the database. The source code checks whether the delete parameter is received using `isset($_GET['delete'])`. After that, the room ID is retrieved and converted into an integer using `intval()` to ensure safer data processing. The system then executes the SQL query `DELETE FROM m_room WHERE room_id = $id` to remove the selected room record from the database. Once the deletion process is completed successfully, the system redirects the user back to the main room page using `header('Location: index.php')`. This feature helps administrators manage room records more efficiently by deleting unused or unnecessary room data from the system.



I. GITHUB LINK

<https://github.com/rentsaisy/Final-Semester-Exam-for-Database-Systems-Practicum>